PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

FULL STACK WEB DEVELOPMENT – CHRONIC KIDNEY DISEASE

AN INTERNSHIP REPORT

Submitted by

ROHITHA S - 20221CSE0707

Under the guidance of,

Mr. Jerrin Joe Francis

BACHELOR OF TECHNOLOGY

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "**Full Stack Web Development – Chronic Kidney Disease**" is a bonafide work of ROHITHA S (20221CSE0707)**,** who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025-2026**.**

Mr. Jerrin Joe Francis
Project Guide
PSCS
Presidency University

Dr. Balakrishnam
Program Internship Coordinator
PSCS
Presidency University

Dr. Md Ziaur Rahaman
School Internship Coordinator
PSCS
Presidency University

Dr. Asif Mohamed H B
Head of the Department,
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS / PSIS
Presidency University

Name and Signature of the Examiners

Sl. No	Name	Signature	Date
1	Mr. H SenthilKumar		
2	Ms. Attar Mahay Sheetal		

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING , at Presidency University, Bengaluru, named ROHITHA S, hereby declare that the internship work titled "**Full Stack Web Development – Chronic Kidney Disease** " has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Name of the Student: ROHITHA S USN: 20221CSE0707 Signature

PLACE: BENGALURU

DATE: 23 - April 2026

INTERNSHIP COMPLETION CERTIFICATE

रेल पहिया कारखाना
RAIL WHEEL FACTORY
भारत सरकार / Govt. of India
रेल मंत्रालय / Ministry of Railways

महा प्रबंधक का कार्यालय,
कार्मिक विभाग
यलहंका, बैंगलूरु - 560 064
General Manager's Office,
Personnel Department
Yelahanka, Bangalore - 560 064

DATE: 10.04.2026

NO.RWF/NG-16/348/Pt III

CERTIFICATE

This is to certify that **Ms. ROHITHA S, (20221CSE0707)**, B.Tech (Computer Science & Engineering) Student of Presidency University, has underwent Internship on "**Full Stack Web Development – Chronic Kidney Diseases**" **at Rail Wheel Factory, Yelahanka, Bangalore** for a period of 03 months i.e., 06th January 2026 to 06th April 2026.

The student has shown keen interest in learning during the period she was with us for the Internship. We found her Enthusiastic, Diligent, Hard Working and Creative.

We wish a bright future and success in all her endeavors.

K.S. Kiran
(K.S.KIRAN)
SENIOR PERSONNEL OFFICER-I

Tel : +91-80-28072402 Fax : +91-80-28460306

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Mr. Jerrin Joe Francis**, Presidency School of Computer Science and Engineering, Presidency University, for his/her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Asif Mohamed H B, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his/her mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Dr. Balakrishnam,** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

ROHITHA S

ABSTRACT

Chronic Kidney Disease (CKD) is a widespread long-term health condition that often goes unnoticed in its early stages, making it especially dangerous. Many patients do not experience clear symptoms until the disease has significantly progressed, which highlights the importance of early detection in preventing severe kidney damage and improving outcomes. To address this challenge, a web-based CKD prediction system was developed using machine learning techniques to provide quick and reliable risk assessments based on basic clinical data. The system is built on the CKD dataset, which includes important health indicators such as blood pressure, specific gravity, albumin levels, red blood cell count, hemoglobin, and other biochemical parameters. A comprehensive data preprocessing pipeline was implemented to ensure model effectiveness, including handling missing values, encoding categorical variables, and normalizing continuous features. For predictive modeling, two widely used machine learning algorithms—Logistic Regression and Random Forest—were trained and evaluated.

The results showed that the Random Forest model significantly outperformed Logistic Regression, achieving around 100% accuracy compared to approximately 88%. This demonstrates the strength of ensemble methods in capturing complex patterns within medical data. To make the solution practical and user-friendly, the trained Random Forest model was integrated into a full-stack web application. The backend was developed using Flask to manage prediction logic, while the frontend was built with React to provide an intuitive interface. Through this platform, users can input patient data and instantly receive CKD risk predictions. Overall, the project showcases how machine learning can enhance early diagnosis, support clinical decision-making, and contribute to better healthcare outcomes.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Acknowledgement	ii
	Abstract	iii
	List of Figures	vi
	List of Tables	vii
	Abbreviations	viii
1.	Introduction 1.1 Background 1.2 Statistics of Internship 1.3 Prior existing technologies 1.4 Proposed approach 1.5 Objectives 1.6 SDGs 1.7 Overview of internship report	1-6
2.	Literature Review	7-10
3.	Methodology	11-21
4.	Project Management 4.1 Internship timeline 4.2 Risk analysis 4.3 Internship budget	22-29
5.	Analysis and Design 5.1 Requirements 5.2 Block Diagram 5.3 System Flow Chart 5.4 Choosing devices 5.5 Designing units 5.6 Standards	30-46

	5.7 Mapping with IoTWF reference model layers 5.8 Domain model specification 5.9 Communication model 5.10 IoT deployment level 5.11 Functional view 5.12 Mapping IoT deployment level with functional view 5.13 Operational view 5.14 Other Design	
6.	Hardware, Software and Simulation 6.1 Hardware 6.2 Software development tools 6.3 Software code 6.4 Simulation	47-53
7.	Evaluation and Results 7.1 Test points 7.2 Test plan 7.3 Test result 7.4 Insights	54-66
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social aspects 8.2 Legal aspects 8.3 Ethical aspects 8.4 Sustainability aspects 8.5 Safety aspects	67-72
9.	Conclusion	73
	References	74-75
	Base Paper (Optional)	75
	Appendix	76-78

LIST OF FIGURES

Sl. No.	Figure Name	Caption	Page No.
1	F 1.1	Sustainable Development Goals	5
2	Figure 3.1	The V Model Methodology	13
3	Figure 3.2	The Devops	14
4	Figure 3.3	SDLC Phases	17
5	Figure 3.4	Summary Of Project Breakdown To Task	22
6	Figure 4.1	PESTEL Analysis	25
7	Figure 4.2	Project Phase Risk Matrix	26
8	Figure 5.1	Functional Block Diagram	34
9	Figure 5.2	System Flow Chart	35
10	Figure 5.3	Example Of Interfacing A One Unit	38
11	Figure 5.4	Domain Model For Home Automation	41
12	Figure 5.5	Communication Model	42
13	Figure 5.6	IOT Deployment Level Suitable For CKD	42
14	Figure 5.7	Functional View For CKD	45
15	Figure 5.8	Operational View	47
16	Figure 7.1	Class Distribution	64
17	Figure 7.2	Correlation Heatmap Of CKD Features	65
18	Figure 7.3	Pairwise Feature Distributiom For CKD And Non CKD	66

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 2.1	Summary Of Literature Reviews	11-12
2	Table 3.2	Mapping Of SDLC To CKD Project	20-21
3	Table 4.1	Project Planning Timeline	23
4	Table 4.2	Project Implementation Timeline	24
5	Table 4.3	Potential Risks In Project	26-27
6	Table 4.4	Project Budget	27
7	Table 5.1	CKD Prediction System Requirements	31
8	Table 5.2	Comparison Of Processing Platforms	38
9	Table 5.4	Mapping Project Layers With IOTWF Reference Model	40
10	Table 5.5	Description Of Domain Model	41
11	Table 7.1	Test Plan	58-59
12	Table 7.2	Test Values At Each Test Point	61
13	Table 7.3	Sample Test Results For Various Input Cases	63-64

ABBREVIATIONS

Abbreviation	Full Form
CKD	Chronic Kidney Disease
ML	Machine Learning
RF	Random Forest
BP	Blood Pressure
BGR	Blood Glucose Random
SC	Serum Creatinine
RBC	Red Blood Cells
WBC	White Blood Cells
API	Application Programming Interface
UI	User Interface
DB	Database

Chapter 1

INTRODUCTION

Chronic Kidney Disease (CKD) is a progressive medical condition in which the kidneys gradually lose their ability to filter blood effectively. One of the major challenges associated with CKD is that it often shows minimal or no symptoms in its early stages, leading to late diagnosis when a significant portion of kidney function—often more than 60%—has already been lost. This delay in detection increases the risk of severe complications such as heart disease, stroke, and eventual kidney failure. Therefore, early diagnosis and screening play a crucial role in improving patient outcomes and supporting timely clinical decision-making .

With the advancement of Artificial Intelligence (AI) and Machine Learning (ML), data-driven decision support systems are becoming increasingly important in healthcare. ML models can efficiently analyze medical data, identify hidden patterns, and provide accurate disease predictions faster than traditional methods. Various studies have demonstrated the effectiveness of classifiers such as Logistic Regression, Support Vector Machines (SVM), and Random Forest in detecting CKD with high accuracy. In this project, an ML-based predictive system is developed to enable early detection of CKD through a web-based application. The backend, implemented using FastAPI, handles model execution and API communication, while the frontend, developed using ReactJS, provides a user-friendly interface. The system takes clinical parameters such as haemoglobin, blood pressure, glucose level, serum creatinine, and blood urea as input, and outputs the prediction along with a confidence score and risk level.

1.1 Background

CKD is a significant health concern in the world due to the increasing lifestyle diseases, blood pressure, diabetes, and absence of routine medical examination. In research studies, it has been established that CKD affects millions of people in the world without their knowledge since it is a silent disease.

The traditional diagnosis will involve clinicians manually reading various parameters of the laboratory, interpreting each of these features, and then determining whether a patient

could be having CKD. Nonetheless, this manual system is time consuming, highly reliant on doctor knowledge and it is susceptible to human error. Medical diagnostics have demonstrated high potential when it comes to machine learning technologies. Other research has shown that ML algorithms are effective in detecting CKD patterns with the help of patient datasets. To illustrate this point, Singh et al. have found high accuracy with Random Forest in classifying CKD, and Jha et al. have compared a set of ML models and have emphasized that SVM and Random Forest are of very high accuracy in predicting CKD. Hybrid ML methods to increase the reliability of prediction were studied by other researchers.

This project is based on these previous works and leads to the creation of an entire ML-controlled diagnostic system that would combine both predictive modelling and a real-time web interface to make it accessible and practical in a clinical setting.

1.2 Statistics

Other studies have highlighted the increasing incidence of CKD and the need to have early screening instruments. Medical research suggests that CKD occurs in a large number of members of the population, particularly in places where there is limited checkups and delays in the process of diagnosis is prevalent. Early identification would help avoid and delay kidney disease, lower medical expenses and increase the survival rates of patients. The dataset utilized in this project consists of 400 records of patients who have 24 clinical attributes as utilized in other CKD research studies. Such characteristics are the blood pressure, haemoglobin, packed cell volume, blood urea, serum creatinine, and diabetic indicators. Both CKD and non-CKD labelled cases are present in the dataset, which is appropriate to use in classification tasks.

Statistical analysis of this dataset indicates that there are evident differences in parameters like haemoglobin, serum creatinine between CKD and non-CKD patients and this enhances the impetus to design automated prediction algorithms to aid in the early detection.

1.3 Prior existing technologies

A number of earlier systems and studies have used machine learning to predict CKD: ML-Based Diagnostic Models ML algorithms have been widely used to make predictions

of CKD: Random Forest and Logistic Regression models have shown good classification ability. When feature selection and normalization methods are done properly, SVM has performed well. Multifunctional models based on the combination of several ML algorithms have also been suggested to enhance the reliability .

Deep Learning Approaches Deep Learning has already been investigated in predicting CKD, where they have demonstrated good results but with large datasets and high computation costs, which cannot be used in lightweight systems. Web-Based CKD Forecasting Applications. Other studies have employed straightforward web-based applications based on such frameworks as Flask to predict CKD in real-time.

Nevertheless, the previous applications did not have: User-friendly design, Scalable backend architecture, Clear interpretability, and Real time, visualization capability. The suggested system eliminates these shortcomings by incorporating ML, FastAPI, and ReactJS to bring more performance and user-friendliness.

1.4 Proposed approach

Aim of the Project to come up with an accurate, reliable, and real-time machine learning-based system of early Chronic Kidney Disease detection on the basis of clinical information. Motivation Different diagnosis of CKD is mainly made late when the damage is usually irreversible. Diagnostic processes could be subject to error and delays, particularly manual ones. ML has the ability to draw out underlying associations in medical information that a clinician can overlook. There is accessibility through a web-based interface to the hospitals and remote users.

Proposed Methodology The steps included in the proposed system are the following:

- Preprocessing of datasets: Processing of missing values, encoding of categorical features and scaling numerical features.
- ML model training: Logistic Regression, SVM, and Random Forest.
- Model selection: The results of the evaluation revealed that the highest accuracy was 100% of Random Forest .
- System development: The selected model is integrated into FastAPI backend. ReactJS frontend will enable the entry of patient data and show predictions.
- Generation of results: Prediction (CKD/No CKD), confidence score and risk level are provided. Applications Dialysis centers and hospitals.

Telemedicine platforms Medical diagnostic labs Health screening camps Research

institutions Limitations. The size of the dataset is also small (400 records) and this influences generalization of the model. It does not use deep learning models because of the limitation of the dataset. Medical privacy standards have to be considered when incorporating real patient data. Application of performance in new populations without retraining may differ.

1.5 Objectives

The functional and quantifiable aims of this project are:

- To classify and analyze the patient medical data with ML algorithms to enable accurate CKD prediction.
- To apply practical pre-processing techniques to address missing data, categorical data as well as inconsistent data.
- To develop a secure interactive web interface that will enable easy CKD prediction.
- To be able to make real-time model inferences using a FastAPI backend that will be able to handle data efficiently.
- To roll out a complete system that combines ML, API services and a web interface for healthcare-related applications.

1.6 SDGs

The project is in accordance with the following UN Sustainable Development Goals shown in figure 1.1:

SDG 3 – Good Health and Well-Being

The proposed system significantly contributes to ensuring healthy lives and promoting well-being by enabling the early detection of Chronic Kidney Disease (CKD). Early diagnosis plays a crucial role in preventing the progression of kidney damage, reducing complications, and lowering mortality rates associated with the disease. By analyzing key medical parameters and providing quick predictions, the system assists healthcare professionals in making timely and informed decisions. This not only improves patient outcomes but also reduces the burden on healthcare infrastructure by minimizing late-stage treatments such as dialysis or kidney transplantation. Furthermore, the system supports preventive healthcare by encouraging

regular health monitoring, thereby aligning closely with the goals of SDG 3 to enhance overall public health and quality of life.

SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates strong alignment with SDG 9 by integrating advanced technologies such as machine learning models, FastAPI-based backend services, and a user-friendly web interface to create an innovative healthcare solution. It showcases how modern digital infrastructure can be leveraged to address critical medical challenges efficiently and cost-effectively. The system represents a step toward the digital transformation of healthcare, where intelligent tools assist in diagnosis and decision-making. Additionally, the scalable and web-based nature of the platform ensures that it can be expanded and integrated into larger healthcare systems in the future. By promoting technological innovation and building robust digital solutions, the project contributes to strengthening healthcare infrastructure and fostering sustainable industrial development in the medical domain.

Fig 1.1 Sustainable development goals

1.7 Overview of project report

The report on the project is divided into nine chapters. **Chapter 1** gives a general idea of the project by sharing its background, explaining the necessity of early CKD prediction, listing objectives, showing how SDG is aligned with the project and describing the proposed approach. The **Chapter 2** is the literature review where the author summarizes and compares the findings of the last few years' conferences and journals on machine learning techniques for CKD detection. **Chapter 3** deals with the method applied in the project which includes the chosen SDLC model, workflow steps, and visual representations. The **Chapter 4** discusses the managing of the project in terms of Gantt chart, risk analysis using PESTEL, risk matrices, and budgeting. The **Chapter 5** is about system analysis and design with topics such as requirements, block diagrams, flowcharts, communication model, domain model, functional views, and deployment mapping discussed. **Chapter 6** includes the various tools used in software development, the configuration procedures, and the implementation environment for building and testing the machine learning model. In visualizations, performance metrics, and insights, **Chapter 7** gives the evaluation and the results obtained from the model. **Chapter 8** debates the social, legal, ethical, sustainability, and safety issues raised by deploying a medical machine learning model. The project is wrapped up in **Chapter 9** where the objectives met, results obtained, and suggestions for future improvements are presented.

Chapter 2

LITERATURE SURVEY

A literature survey is a study of all the information and research that already exists about a topic. It helps us understand what other researchers have done, what methods they used, and what results they found. By reading previous work, we can identify what is already known and what gaps still exist. This allows us to build our project or research on a strong foundation and avoid repeating the same work. In short, a literature survey gives a clear background of the topic and shows why our study is important.

Based on basic learning algorithm

- Singh et al. [1] presented a CKD prediction model using the Random Forest algorithm, utilizing clinical parameters such as blood pressure, hemoglobin, and serum creatinine. The study demonstrated that Random Forest effectively captures non-linear relationships in medical data, achieving high accuracy and stable performance. However, the model's effectiveness depends heavily on the quality and representativeness of the training dataset, which may limit its generalizability.
- Jha et al. [2] conducted a comparative analysis of multiple machine learning algorithms including Support Vector Machine (SVM), Logistic Regression, and K-Nearest Neighbors (KNN) for CKD prediction. Their findings emphasized the importance of data preprocessing in improving model performance. While SVM achieved higher precision, Logistic Regression provided better interpretability. However, some models showed sensitivity to noisy and inconsistent data.
- Al Imran and Rahman [3] proposed a hybrid framework combining Logistic Regression and Random Forest. Logistic Regression was used for feature analysis, while Random Forest performed classification. This integrated approach improved prediction balance and robustness, though it introduced additional complexity in parameter tuning and model optimization.
- Wahab et al. [4] introduced a hybrid model combining Gradient Boosting and Random Forest techniques. By integrating boosting and bagging strategies, the model effectively addressed class imbalance and achieved high predictive accuracy. However, the approach involves increased computational complexity and higher processing time.

- Elbattah and Soliman [5] developed a deep learning-based CKD detection model capable of automatically extracting complex patterns from high-dimensional medical data. While the model showed strong predictive capability, it requires large datasets and significant computational resources for optimal performance.
- Lakshmi and Reddy [6] explored the use of Decision Tree algorithms for CKD detection, providing an interpretable and rule-based classification approach suitable for clinical use. However, the model is prone to overfitting, especially when applied to smaller or imbalanced datasets.
- Verma and Tiwari [7] utilized the Naive Bayes classifier for CKD prediction, highlighting its simplicity and computational efficiency. Although it enables fast predictions, its assumption of feature independence may reduce accuracy in complex medical datasets where feature relationships exist.
- Kumar and Yadav [8] investigated the application of Artificial Neural Networks (ANN) for CKD risk analysis. The multi-layer architecture allows the model to capture complex non-linear relationships between clinical features. However, its black-box nature limits interpretability, which is a critical concern in healthcare applications.
- Chen and Zhang [9] applied the XGBoost algorithm for CKD prediction, demonstrating high efficiency and accuracy by iteratively improving weak learners. Despite its strong performance, the model requires careful parameter tuning and involves higher computational cost.
- Patel and Shah [10] implemented the K-Nearest Neighbors (KNN) algorithm for CKD classification. The approach is simple and intuitive, relying on similarity measures between data points. However, it becomes computationally expensive and less efficient when handling large datasets.
- Kang et al. [11] conducted a comprehensive study on various machine learning techniques for CKD prediction, emphasizing the importance of feature selection and preprocessing. Their results showed that optimized models can achieve high reliability, though the process requires significant effort in data preparation and tuning.
- Chittora et al. [12] proposed a CKD prediction framework using multiple machine learning models such as SVM, Random Forest, and Logistic Regression. The study highlighted the effectiveness of ensemble learning and feature importance analysis in improving prediction accuracy. However, integrating multiple models increases system complexity and computational requirements.

Summary of Literatures reviewed:

Table 2.1 Summary of Literature reviews

Sl	Article Title, Published Year, Journal Name	Authors	Methods	Key Features	Merits	Demerits
1	CKD Prediction Using Random Forest, 2020, Procedia Computer Science	S. Singh, P. Sharma, R. Kumar	Random Forest	Uses BP, hemoglobin, creatinine; handles non-linear data	High accuracy, stable	Dataset dependent
2	Comparative Study of ML Models, 2020, IJERT	G. Jha, R. Arora, N. Sinha	SVM, LR, KNN	Model comparison; preprocessing importance	High precision (SVM), interpretable (LR)	Sensitive to noise
3	CKD Prediction using Logistic Regression and Random Forest, 2019, IJCA	S. Al Imran, M. Rahman	Hybrid (LR + RF)	LR for feature analysis, RF for classification	Balanced performance	Complex tuning
4	Hybrid Model for CKD Prediction, 2020, IEEE Access	M. A. Wahab, A. Hossain, M. Rahman	GB + RF	Boosting + bagging; handles imbalance	Very high accuracy	High computation
5	Data-Driven Approach for CKD Detection, 2019, Springer	R. M. Elbattah, A. Soliman	Deep Learning	Learns complex patterns	Strong accuracy	Needs large data
6	CKD Detection using	K. R. Lakshmi	Decision Tree	Rule-based classification	Easy to interpret	Overfitting issue

	Decision Tree, 2018	, M. V. Reddy				
7	CKD Prediction using Naive Bayes, 2017	A. K. Verma, S. Tiwari	Naive Bayes	Probabilistic approach	Fast and simple	Lower accuracy
8	CKD Risk Analysis using ANN, 2021	M. Kumar, S. Yadav	Neural Network	Multi-layer learning	Captures complex relations	Black-box model
9	CKD Prediction with XGBoost, 2022	L. Chen, H. Zhang	XGBoost	Advanced boosting technique	High performance	Computational cost
10	CKD Classification using KNN, 2019	P. R. Patel, D. Shah	KNN	Instance-based learning	Simple implementation	Slow for large data
11	Machine Learning Approaches for CKD Prediction, 2021, Scientific Reports	M. Kang, J. Kim, E. Park	Multiple ML Models	Comparative analysis; feature selection	High reliability	Requires preprocessing
12	Prediction of CKD using ML Techniques, 2021, IEEE Access	P. Chittora, S. Patil, A. Kumar	SVM, RF, LR	Ensemble learning; feature importance	Improved accuracy	Model complexity

Research Gaps

Gap1:

Despite the extensive research carried out in CKD prediction, a significant gap exists in the practical implementation of these models. Studies such as [1], [2], and [6] primarily concentrate on evaluating the performance of various machine learning algorithms and achieving high prediction accuracy. However, these works are largely limited to theoretical analysis and experimental validation, with minimal focus on real-time deployment in user-friendly systems. Although [8] introduces a web-based application, the integration of highly accurate models with scalable, responsive, and easily accessible platforms remains insufficient. This creates a gap between model development and real-world clinical usability, where healthcare professionals require fast, reliable, and interactive systems for decision-making.

Gap2:

Another major gap observed in the literature, particularly in studies like [3], [5], and [10], is the trade-off between model accuracy and system complexity. Many approaches achieve high performance through advanced algorithms, extensive preprocessing, and parameter tuning, which increases computational overhead and reduces efficiency. Such complexity can limit their adoption in real-time healthcare environments, especially in resource-constrained settings. Additionally, these models often lack simplicity and ease of interpretation, which are crucial for clinical acceptance. Therefore, there is a clear need for a balanced system that maintains high accuracy while ensuring low computational complexity, faster processing, and practical usability in real-world applications.

Chapter 3

Methodology

The building of the Chronic Kidney Disease (CKD) Prediction System was done according to the Software Development Life Cycle (SDLC) methodology. SDLC offers a residue framework that practically leads the complete software development process, from requirement gathering to system deployment and maintenance. The application of this structured approach guarantees that every activity will be organized, tracked, and executed in a systematic manner to produce a top-quality system. The SDLC model is appropriate for this project as the CKD prediction system has numerous technical components such as data management, model creation, web-based user interface, back-end API connection, and testing. All these components can easily be mapped to the different phases of SDLC making the development flow smooth and well-organized.

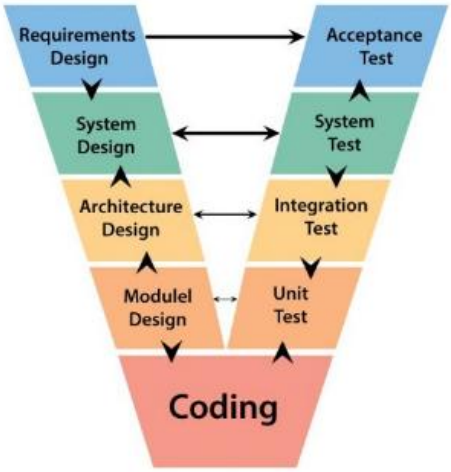


Fig 3.1 The V model methodology

Requirement Gathering:

The project started with requirement gathering, where the main objectives were identified, such as predicting Chronic Kidney Disease (CKD) using patient inputs like blood pressure and haemoglobin, ensuring a user-friendly interface, and providing quick and accurate predictions. These requirements guided the development process and defined the system's goals. At the end, acceptance testing was carried out to verify whether the final system met all

user expectations, including accuracy of predictions, proper user interface design, and correct output generation.

Analysis:

During system analysis, the dataset was carefully studied, and suitable machine learning models such as Random Forest and Logistic Regression were selected. The overall workflow of the system was also defined, including how data flows from input to prediction. Corresponding to this phase, system testing was performed on the complete application to ensure that all components—frontend, backend, and machine learning model—worked together smoothly without errors.

Design:

In the system design phase, the architecture of the application was planned by defining the use of a React frontend, a Flask/FastAPI backend, and integration of the machine learning model. This design ensured a structured and scalable system. Integration testing was then conducted to verify that all parts of the system communicated effectively, particularly the interaction between the frontend interface, backend APIs, and the prediction model.

Module:

The system was further divided into smaller modules during the module design phase, including data preprocessing, model prediction, API handling, and user interface components. This modular approach made development and debugging easier. Each module was individually tested through unit testing to ensure that every part functioned correctly, such as validating prediction functions and checking API responses.

Coding:

In the coding phase, the entire system was implemented using appropriate technologies. Python with Flask or FastAPI was used for backend development, React was used for building the frontend interface, and machine learning models were integrated for CKD prediction. This phase brought together all the planned components into a fully functional and working application.

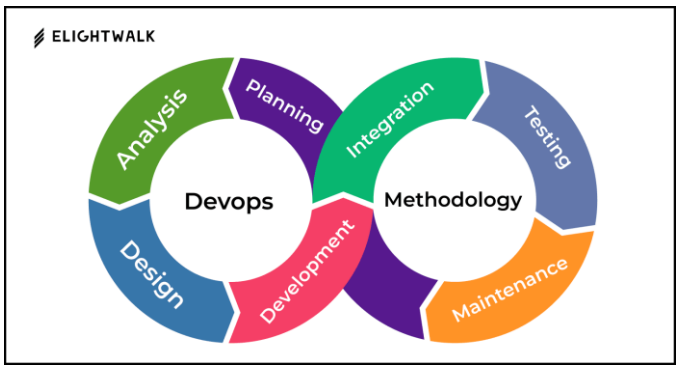


Fig 3.2 The Devops methodology

Planning

The project began with identifying key goals such as achieving high accuracy in Chronic Kidney Disease (CKD) prediction and designing a user-friendly interface. Decisions regarding dataset usage, selection of suitable models, and defining important system features were also made during this phase.

Analysis

In this phase, the CKD dataset was thoroughly analyzed to understand important medical parameters like blood pressure, haemoglobin, and serum creatinine. System requirements related to model performance and overall functionality were also carefully studied.

Design

The system architecture was designed by defining components such as the React frontend, Flask/FastAPI backend, and the machine learning model. The user interface layout and data flow for prediction were also planned to ensure smooth operation.

Development

During development, machine learning models were implemented along with backend APIs. At the same time, the frontend interface was developed to allow users to input patient data and view prediction results effectively.

Integration

In the integration phase, all components including the frontend, backend, and trained model were combined into a single system. Proper communication between these components was ensured for seamless functionality.

Testing

The system was thoroughly tested to evaluate model accuracy, API responses, and user interface performance. Bugs were identified and fixed to ensure the system works correctly under different conditions.

Maintenance

After deployment, the system was continuously monitored to ensure optimal performance. Updates and improvements were made over time by incorporating new data and refining the model for better accuracy.

3.1 SDLC Phases

The SDLC comprises of seven primary phases as shown in figure 3.1:

Analysis:

In this phase, all functional and non-functional requirements of the system are carefully gathered and documented to ensure a clear understanding of the project. This includes identifying stakeholder needs, defining the scope of the project, understanding constraints such as time, cost, and resources, and setting realistic expectations. Functional requirements specify what the system should do, such as predicting Chronic Kidney Disease (CKD) based on patient inputs, while non-functional requirements focus on aspects like performance, usability, security, and scalability. Proper requirement analysis helps avoid misunderstandings later in the project and provides a strong foundation for all subsequent phases.

Design:

Based on the gathered requirements, the system design phase creates a detailed blueprint for development. This includes both high-level design, which outlines the overall system architecture, and low-level design, which focuses on individual components and their interactions. Key elements such as database schema design, user interface (UI) and user experience (UX) planning, and selection of appropriate technologies (e.g., React for frontend, Flask/FastAPI for backend) are finalized. Data flow diagrams, architecture diagrams, and module designs are also prepared. This phase ensures that the development team has a clear structure to follow, reducing confusion and improving efficiency.

Implementation:

During the implementation phase, developers begin writing the actual code based on the design specifications. The system is divided into modules such as data preprocessing, model prediction, API handling, and user interface components, which are developed individually and then integrated. Coding standards and best practices are followed to ensure maintainability and scalability. Regular code reviews and version control are used to track changes and maintain quality. This phase transforms the theoretical design into a working software system.

Testing:

Once development is complete, the system undergoes rigorous testing to identify and fix any defects or inconsistencies. Various types of testing are performed, including unit testing to verify individual components, integration testing to ensure modules work together correctly, system testing to validate the entire application, and user acceptance testing to confirm that the system meets user requirements. Testing also checks for performance issues, security vulnerabilities, and usability concerns. This phase is crucial to ensure the software is reliable, accurate, and ready for real-world usage.

Deployment:

After successful testing, the system is deployed to the production environment where it becomes accessible to end-users. This phase may involve pilot deployment to a limited group of users for final validation, followed by a full-scale release. Activities such as server setup, configuration, and user training are also carried out. Proper deployment ensures that the system runs smoothly in a real-world environment and that users can effectively utilize its features.

Maintenance:

Following deployment, the system enters the maintenance phase, where continuous support and improvements are provided. This includes fixing bugs that may arise during real-world usage, optimizing performance, updating the system to adapt to new requirements, and adding new features based on user feedback. Regular monitoring is done to ensure system stability and efficiency. Maintenance helps in extending the lifespan of the software and keeping it relevant over time.

Post-Implementation:

In this final phase, the overall performance and effectiveness of the system are evaluated after deployment. Feedback from users and stakeholders is collected to assess whether the system meets its intended objectives. Performance metrics, system reliability, and user satisfaction are analyzed to identify areas for improvement. This phase may also involve planning future enhancements and long-term monitoring strategies. A thorough post-implementation review ensures continuous improvement and helps in making the system more robust and efficient in the long run.

These stages are graphically presented in

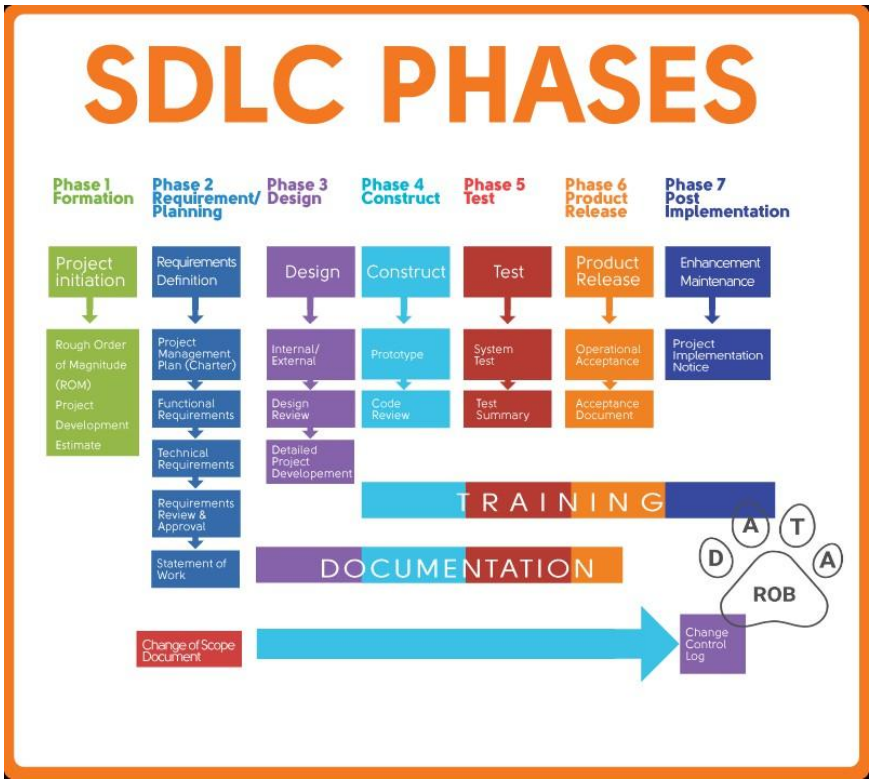


Fig 3.3 SDLC phases

The sequential flow of the SDLC model, as depicted in Fig 3.1, starts from planning and includes analysis, design, implementation, testing, deployment, and maintenance. The model has the feature of structured development and guarantees that every stage is assessed before proceeding to the next.

3.1.1 Requirement Analysis

During this phase, the requirements of the CKD prediction system, both functional and non- functional, were recognized.

1. Functional Requirements

• Reception of clinical inputs from patients (BP, haemoglobin, glucose, serum creatinine, etc.)
• Conducting machine learning-based prediction (CKD or non-CKD)
• Displaying confidence score and risk level of prediction
• Saving and retrieving previous prediction records
• Providing a user-friendly interface for medical staff and patients
• Generating downloadable reports (PDF/CSV) for predictions
• Displaying model performance metrics such as accuracy and ROC-AUC
• Enabling API communication between frontend and backend for data exchange
• Providing alerts for high-risk cases or abnormal patient values

2. Non-Functional Requirements

• High accuracy and reliability of prediction results
• Protection and confidentiality of patient data
• Secure data transmission between frontend and backend
• Easy maintenance and modular system design
• Scalability to handle more users, datasets, and features in future
• High system availability with minimal downtime
• User-friendly interface for better usability
• Compatibility across different devices (desktop, mobile, tablet)
• Efficient memory and resource utilization
• Fast loading time for UI and API responses

This analysis made it easier to understand the needed features and the expected system behaviour.

3.1.2 System Design

The design of the system included the making of architectural diagrams, database structures, user interface layouts, and identification of required tools.

The project utilizes a combination of modern technologies to ensure efficient development and performance. ReactJS is used for the frontend to build an interactive and user-friendly interface, allowing users to easily input patient data and view prediction results along with visual insights. For the backend, FastAPI is implemented due to its high performance and ability to handle API requests quickly, enabling smooth communication between the frontend and the machine learning model. Python, along with the scikit-learn library, is used to develop and train the machine learning models, providing reliable tools for data preprocessing, model training, and evaluation.The system is built using the UCI Chronic Kidney Disease dataset, which contains real medical records and serves as the foundation for training and testing the prediction model. For data visualization, libraries such as Recharts and Material UI are used to create interactive charts and a visually appealing interface, helping users better understand patterns and results. Additionally, Draw.io is used to design system architecture and workflow diagrams, which assist in clearly representing the structure and functioning of the overall system. Together, these technologies contribute to building a robust, scalable, and easy-to-use CKD prediction system.

3.1.3 Implementation / Development

The whole system was implemented in two's major parts during this phase: In this phase, the machine learning model for CKD prediction was developed through a series of structured steps. Initially, dataset preprocessing was performed, which included handling missing values, encoding categorical variables, and normalizing numerical features to ensure consistency. The dataset was then divided into training and testing sets to evaluate model performance effectively. Three different models—Logistic Regression, Support Vector Machine (SVM), and Random Forest—were trained and compared. Among these, Random Forest was selected as the final model due to its highest accuracy of 100% and strong performance in handling complex data patterns. The trained model was then saved as a `.pkl` file, making it easy to reuse for prediction without retraining.

Application:

The web application was developed by combining a React JS frontend with a Fast API backend to create a complete and interactive system. On the frontend, a user-friendly interface was designed to allow users to input clinical data, view prediction results, and access a dashboard with charts and visual analysis. On the backend, Fast API was used to build an endpoint (`/predict`) that receives input data, processes it through the trained machine learning model, and returns the prediction along with a confidence score. Both frontend and backend were integrated seamlessly to ensure smooth communication, enabling real-time predictions and an efficient user experience.

3.1.4 Testing

Testing was the means to validate that the system behaves properly. Types of Testing Done **Unit Testing**: It was performed on deploy including ML model functions, backend API logic, and frontend components.

Integration testing confirmed that the React JS frontend and Fast API backend worked seamlessly together, enabling smooth data flow from user input to API processing, model prediction, and final output display.

System testing ensured that the entire application functioned correctly, covering prediction accuracy, user interface behavior, and proper error handling.

Validation was performed by comparing predicted results with outputs from the test dataset, confirming the reliability of the model. The Random Forest classifier was verified as the final model for prediction, demonstrating strong performance and consistency. Overall, the CKD prediction system proved to be accurate, stable, and ready for deployment.

3.1.5 Maintenance

Maintenance covers the following:

The maintenance phase of the SDLC ensures the system remains relevant and effective over time by continuously improving and updating it. This includes retraining the model whenever new patient datasets become available to maintain accuracy, adding new features to enhance the frontend, and refining the overall user experience based on user feedback. It also involves

identifying and fixing bugs or performance issues to ensure smooth system operation. Additionally, feature expansion such as storing patient history and generating visual reports helps in improving functionality and usability. Overall, this phase focuses on sustaining the system's performance, adaptability, and long-term reliability.

3.2 Mapping of SDLC to CKD Project

Table 3.1 Mapping of SDLC to CKD project

SDLC Phase	**Mapped to CKD Project Task**
Requirement Analysis	Identifying necessary clinical parameters and system functions
System Design	Architecting ML pipeline, API structure, frontend forms
Development	Training ML model, building ReactJS frontend, FastAPI backend
Testing	Unit testing, integration, system-level verification
Deployment	Hosting backend & frontend, connecting ML model
Maintenance	Updating models, adding new features

In Table 3.1, A detailed representation of the Software Development Life Cycle (SDLC) phases corresponding to the tasks that took place in the Chronic Kidney Disease (CKD) prediction project is shown. Each of the SDLC phases is mapped to the activities done during the system development, which guarantees a systematic and orderly project workflow. In the Requirement Analysis phase, not only were the essential clinical parameters defined which were needed for predicting CKD but also the functional requirements of the system were listed. The System Design phase saw

Frontend input forms. During the Development phase, the ML model was trained, the ReactJS-based frontend was developed, and the FastAPI backend was created. The Testing phase was dedicated to unit testing, integration testing, and checking the entire system's functionality. The Deployment phase comprised of hosting the backend and frontend, and the integration of the ML model with the live system. The Maintenance phase at last, keeps the system continuously improved through the updates of the ML model and the addition of new features whenever necessary.

Fig 3.2 Project Breakdown

Fig 3.2 illustrates the breakdown of a project into manageable tasks such as idea creation, development, execution, and final handover.The project management breakdown illustrates each task's part in the final project execution.

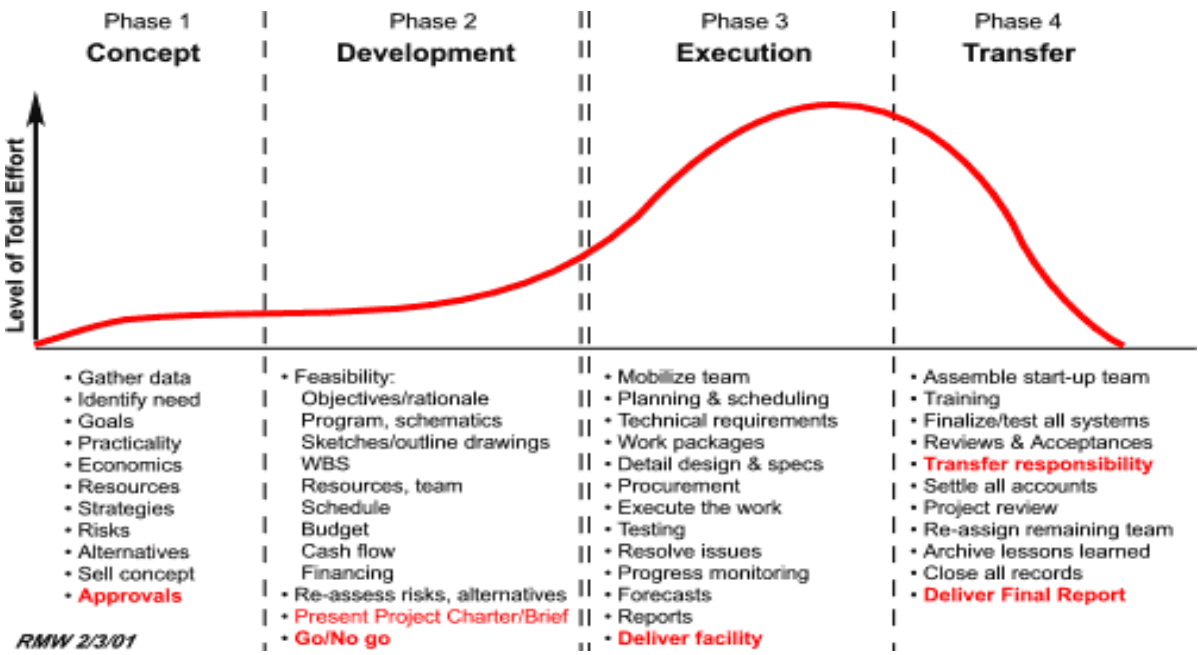


Fig 3.4 Summary of project breakdown to task

Chapter 4

Project Management

4.1 Project Timeline

The complete project management process about CKD Prediction System development consists of planning, scheduling, monitoring, and controlling. A Gantt chart excels in visually representing the week's progressive movement of the project across its different tasks. The phases delineated include project initiation, literature review, system requirement analysis, design, implementation, software development, testing, and report submission. Gantt charts are excellent in pinpointing dependencies, tracking the progress of tasks, and confirming that major developments have occurred within the scheduled time.

Table 4.1 Project Planning Timeline

Major Task	W1	W2	W3	W4	W5	W6	W7
Project Initiation	■	■					
Selection Of Topic	■						
Background Study		■	■				
Objectives Finalization			■	■			
Methodology Selection				■	■		
Proposal Preparation					■	■	
Literature Review						■	■

Description

Week1:

The topic of CKD prediction using machine learning was finalized after evaluating various project ideas. The selection was based on its real-world relevance, impact on healthcare, and the possibility of applying data-driven techniques for early disease detection.

Week(2-3):

During this phase, detailed research was conducted to understand CKD, its causes, symptoms,

and important clinical parameters such as blood pressure, haemoglobin, and serum creatinine. Existing systems and technologies used for disease prediction were also studied to gain domain knowledge.

Week(3–4):

Clear objectives of the project were defined, such as developing an accurate prediction system, ensuring user-friendly interaction, and enabling early detection of CKD. Both functional and performance-related goals were established in this phase.

Week(4-5):

In this phase, the approach for solving the problem was selected. Machine learning techniques were chosen for prediction, and decisions were made regarding the use of algorithms, dataset, preprocessing methods, and evaluation metrics.

Week(5-6):

A formal project proposal was prepared, including problem definition, objectives, methodology, tools, and expected outcomes. This document was structured to present the complete plan of the project for review and approval.

Table 4.2 Project Implementation Timeline

Major Task	W8	W9	W10	W11	W12	W13	W14	W15
System Requirement Phase								
System Design Phase								
Functional Unit Design								
Software Development								
Testing(Unit, Integration, System)								

Critical Evaluation		[redacted]	[redacted]
Final Report		[redacted]	[redacted]

Description

Week(8-9):

During Week 8 and Week 9, the system requirements were gathered and analyzed. This phase focused on identifying both functional and non-functional requirements such as CKD prediction using patient data, system accuracy, fast response time, and user-friendly interface. The inputs, outputs, constraints, and overall system behavior were clearly defined to build a strong foundation for development.

Week(9-10):

In Week 9 and Week 10, the overall architecture of the system was designed. Decisions were made regarding the use of React for the frontend, FastAPI for the backend, and integration of the machine learning model. The data flow between components, UI structure, and API design were planned to ensure smooth communication and efficient performance.

Week(10-11):

During Week 10 and Week 11, the system was divided into smaller functional units such as data preprocessing, machine learning model, backend API, and frontend interface. Each unit was designed separately with clear responsibilities, making the system modular and easier to implement and test.

Week(11-12):

In Week 11 and Week 12, the actual implementation of the system was carried out. The backend APIs were developed using FastAPI, the frontend interface was built using React, and the machine learning model was trained and integrated. All modules were coded according to the design specifications.

Week(12-13):

Week 12 and Week 13 were dedicated to testing. Unit testing was performed on individual components, integration testing ensured proper communication between frontend, backend,

and model, and system testing verified the complete application. Bugs and errors were identified and fixed during this phase.

Week(13-14):

In Week 13 and Week 14, the system was critically evaluated based on performance, accuracy, and reliability. Limitations such as dataset size, computational constraints, and possible improvements were identified. The effectiveness of the model and system usability were analyzed in detail.

Week(14-15):

During Week 14 and Week 15, the complete project documentation was prepared. This included system design, methodology, implementation details, results, evaluation, and conclusions. The final report provides a comprehensive overview of the entire project and its outcomes.

4.2 Risk Analysis

Risk analysis is done via PESTLE analysis and the Project Phase Risk Matrix, which both aid in recognizing external and internal risks that can hinder project success.

PESTEL Analysis

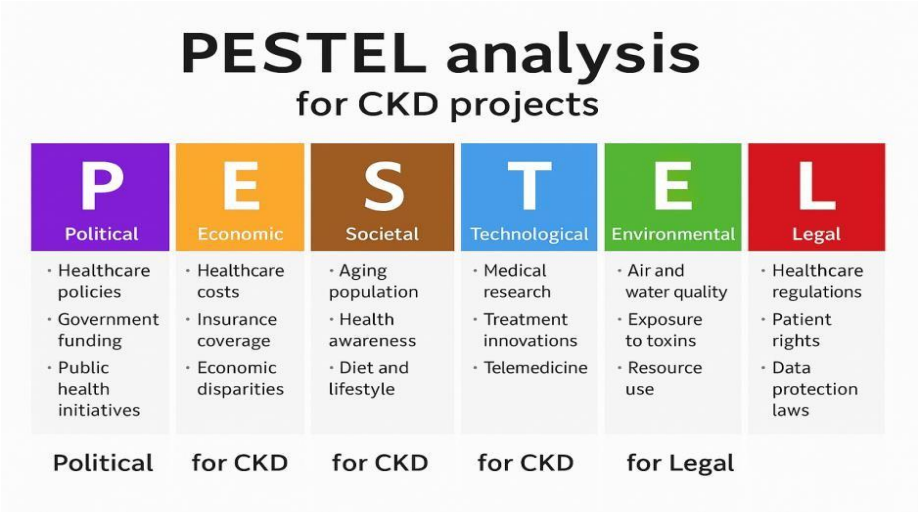


Fig 4.1 PESTEL Analysis

The PESTEL model explains how Political, Economic, Social, Technological, Environmental, and Legal aspects are the factors attributable to the project. In terms of the healthcare-based ML system like CKD prediction:

• **Political:** Government health policies, digital healthcare initiatives, and regulatory approvals can influence how easily the system is deployed in hospitals or clinics. Support from national programs like digital health missions can encourage adoption, while strict regulations may require compliance checks and certifications before implementation.

• **Economic:** The availability of financial resources affects development, deployment, and maintenance of the system. Costs related to infrastructure, cloud services, software tools, and model updates must be considered. Affordable solutions are important for adoption in small clinics and rural healthcare centers.

• **Social:** Acceptance of AI-based diagnosis by both medical professionals and patients plays a major role. Awareness about CKD and trust in technology can impact usage. Proper training, transparency in results, and ease of use can improve user confidence and adoption.

•**Technological:** The system depends on reliable machine learning models, efficient frameworks like React and FastAPI, and proper data preprocessing techniques. Continuous updates, model optimization, and secure data handling are necessary to maintain performance and accuracy. Integration with other healthcare systems can further enhance functionality.

•**Environmental:** Although not directly related, the system's deployment may involve computing resources that consume power. Efficient use of servers, cloud infrastructure, and energy optimization practices can help reduce environmental impact, especially in large-scale deployments.

•**Legal:** Compliance with medical data protection laws in India (such as data privacy regulations and healthcare guidelines) is essential. Patient consent must be obtained before using their data, and strict measures should be taken to ensure confidentiality and prevent misuse. Ethical considerations, such as transparency in predictions and avoiding bias in models, must also be addressed.

Project Phase Risk Matrix

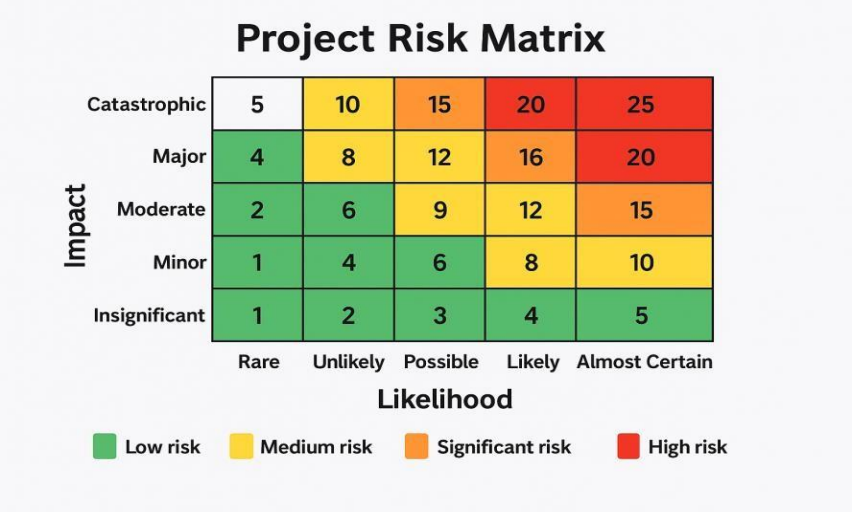


Fig 4.2 Project phase risk matrix

This matrix assesses risk on a probability vs. impact basis:

The Project Phase Risk Matrix displayed in Figure 4.2 sorts risks according to their probability and impact. The matrix gives a score to each possible risk combination, thereby helping in the classification of risks as low, medium, significant, or high risk. Higher scores (e.g., catastrophic and almost certain) are assigned to risks that require urgent action, whereas lower ones are for those that are not a concern at all. This matrix plays a key role in deciding which risks in the CKD project will be followed or lessened in influence during the development process.

Potential Risks in this project

Table 4.3 Potential risks in project

Risk Type	**Impact**	**Probability**	**Risk Level**	**Mitigation**
Incorrect ML Prediction	Major	Medium	High	Retrain with more data; feature engineering
Data Privacy Concerns	Catastrophic	Low	Medium	Secure storage; encrypted transmission
System Downtime	Moderate	Medium	Medium	Use reliable hosting; error handling

Model Over fitting	Moderate	High	High	Cross-validation; regularization
UI/UX Failures	Minor	Medium	Low	Prototype testing; feedback cycle

Description

The risk matrix is used to identify and prioritize potential issues in the system based on their severity and likelihood of occurrence. It helps determine which risks require immediate attention and which can be monitored over time. High-risk factors, such as model accuracy and data security, are given top priority because they directly impact the reliability and safety of the system. By using the risk matrix, appropriate mitigation strategies can be planned and implemented early, ensuring the system remains dependable, secure, and effective in real-world usage.

4.3 Project Budget

The project budget outlines the expenses of the required resources throughout the development process.

Step-by-step cost planning begins with identifying all the tasks and resources required for the project, including development, testing, deployment, and maintenance activities. For each task, an estimate of the labor hours is calculated to understand the total effort involved. In addition to manpower, the required tools such as software, hosting services, and data storage are also considered, as they contribute to the overall cost. For academic projects, a standard cost per hour (typically ₹100 per hour) is assigned to estimate the value of student effort. Finally, all these factors are combined to calculate the total budget, providing a clear understanding of the project's financial requirements.

Table 4.4 Project budget

S.No	Task	Material Cost (₹)	Labor Hours	Cost per Hour (₹)	Total Cost (₹)
1	Dataset Collection & Cleaning	0	10	0	0
2	Model Development	0	20	0	0
3	Web Frontend Development	0	25	0	0
4	Backend API Development	0	20	0	0
5	Testing (Unit + Integration)	0	15	0	0
6	Deployment & Documentation	0	10	0	0
Total		**₹0**	**100 hrs**	**0**	**₹0**

Chapter 5

Analysis and Design

The two key phases in software system development are analysis and design. The focus of analysis is to clarify the objectives the system must meet, while design determines the ways through which the system will meet the requirements. For the Chronic Kidney Disease (CKD) Prediction System, analysis includes a number of activities: problem domain study, user needs understanding, data characteristics identification, constraints definition, and expected system behaviour determination. Design, however, takes these requirements and translates them into technical specifications including such elements as system architecture, functional blocks, workflows, algorithms, interfaces, and testing strategies. Analysis allows the clear specification of system purpose, behaviour, and requirements while design creates the plan for implementation in a structured manner. The two stages of analysis and design, therefore, must be separated, as each in its way ensures that the final system is accurate, reliable, user-friendly, and technically sound.

5.1 Requirements

- The CKD prediction system is developed to classify the patients as CKD or non-CKD correctly by the use of machine learning techniques.
- Requirements are categorized under Hardware Requirements, Software Requirements, Data Requirements, System Management, Security, and User Interface Requirements.

1. Functional Requirements

• Reception of clinical inputs from patients (BP, haemoglobin, glucose, serum creatinine, etc.)
• Conducting machine learning-based prediction (CKD or non-CKD)
• Displaying confidence score and risk level of prediction
• Saving and retrieving previous prediction records
• Providing a user-friendly interface for medical staff and patients
• Generating downloadable reports (PDF/CSV) for predictions

• Displaying model performance metrics such as accuracy and ROC-AUC
• Enabling API communication between frontend and backend for data exchange
• Providing alerts for high-risk cases or abnormal patient values

2. Non-Functional Requirements

• High accuracy and reliability of prediction results
• Protection and confidentiality of patient data
• Secure data transmission between frontend and backend
• Easy maintenance and modular system design
• Scalability to handle more users, datasets, and features in future
• High system availability with minimal downtime
• User-friendly interface for better usability
• Compatibility across different devices (desktop, mobile, tablet)
• Efficient memory and resource utilization
• Fast loading time for UI and API responses

5.1.1 System Hardware Requirement Phase

Even though the project is purely software-driven, the hardware requirements specify the environment where data will be processed and the ML model will be run.The system begins with identifying the initial conditions required for proper functioning, which include a desktop computer or laptop, a continuous power supply, and an internet connection for tasks such as dataset fetching and API hosting. Next, the input parameters are determined, where patient medical data can be either uploaded or entered manually. These inputs include important clinical values such as blood pressure, haemoglobin, serum creatinine, albumin, specific gravity, blood glucose random, and packed cell volume. These parameters form the basis for predicting the presence of Chronic Kidney Disease.The system then produces outcomes in the form of a binary classification, indicating whether the patient has CKD or not, along with a confidence score representing the reliability of the prediction. In some cases, visualizations may also be provided to help interpret the results. The relationship within the system follows a clear flow where input parameters are processed through the machine learning model to generate predictions, and the backend communicates these results to the frontend interface for

user display. However, certain constraints must be considered, such as the system's ability to handle missing values, ensuring that inputs fall within valid clinical ranges, and the dependency of performance on computational resources. Additionally, the accuracy of the system is highly influenced by the quality and completeness of the dataset used for training.

5.1.2 System Software Requirement Phase

In the system software requirement phase, the initial conditions involve setting up a proper development environment that includes Python along with essential libraries such as scikit-learn, pandas, and numpy for data processing and machine learning operations. The backend is implemented using FastAPI to efficiently handle API requests, while the frontend is developed using ReactJS to provide a responsive and user-friendly interface. These components together establish the technical foundation required for the system to function smoothly. The system accepts input parameters in the form of JSON data submitted through the frontend form, where users enter patient details. The machine learning model is designed to process a fixed set of 24 attributes based on the UCI CKD dataset. The outcomes are then presented in a clear and user-friendly manner on the React interface, ensuring easy interpretation of results. The overall workflow follows a structured flow from frontend to API to machine learning model and back to the frontend with the response. However, certain constraints must be maintained, such as ensuring the API remains available and functional, input data strictly follows the required format as per the backend schema, and the trained model is properly loaded from the pickle file to avoid errors during prediction.

Table 5.1 CKD Prediction System Requirements

Category	**CKD Prediction System Requirements**
Purpose	To build a machine learning system capable of predicting CKD accurately using patient clinical parameters.
Behavior	System accepts clinical inputs → preprocesses data → ML model predicts CKD / Non-CKD → displays output to user.
System Management	System should provide API-based backend processing and ensure proper model loading, stability, and logging.

Data Analysis	Perform preprocessing, missing value handling, encoding, splitting, ML training, evaluation, and prediction.
Application Deployment	Application will be deployed locally (development) with provision for hosting on cloud platforms.
Security	Backend validation, secure API access, prevention of malformed inputs, optional user authentication.
User Interface	Clean web interface accepting medical inputs and showing result and confidence score.

The essential system requirements for the CKD Prediction System are listed in 5.1 Table.The machine learning model to be built is defined as the project objective since it will be capable of predicting CKD accurately based on clinical parameters of the patient. The system behavior includes taking user inputs, doing data preprocessing, and providing CKD/Non-CKD forecasts to the user. It covers backend management consisting of stable model loading and logging, besides the data analysis steps such as preprocessing, encoding, and model training. The table also describes the deployment requirements, security measures for safe API usage, and a simple, user-friendly interface for displaying predictions with confidence scores.

5.1.3 System Hardware Design Phase

In this phase, the system is designed by identifying key functional blocks such as the user device (desktop, laptop, or mobile), browser interface, backend server (local or cloud), and computational resources like CPU or GPU. The process flow starts when the user enters medical values, which are sent to the API endpoint. The backend then activates the machine learning model, performs the prediction, and sends the result back to the user interface. Possible inconsistencies such as incorrect input formats, missing fields, or abnormal clinical values are also considered to ensure proper validation.The interfaces are designed to enable smooth communication, where the web interface sends data in JSON format to the backend, and the backend forwards it to the machine learning model for processing. The system design ensures that hardware requirements are minimal, allowing it to run efficiently even on basic systems. Additionally, an integrated testing plan is developed where the system is tested across different hardware setups, and factors like loading time and prediction speed are evaluated to ensure consistent performance and reliability.

5.1.4 System Software Design Phase

In this phase, the software system is structured by identifying key functional blocks such as the React frontend, FastAPI backend, machine learning model, and preprocessing pipeline. The workflow begins when the user provides input, which is first validated before being sent to the backend API. The backend then performs preprocessing on the data and passes it to the machine learning model for prediction. The result is generated and sent back to the user interface for display. During this process, possible inconsistencies such as incorrect data types, missing fields, API timeouts, and model loading errors are considered and handled to ensure smooth operation.The system interfaces are designed using a REST API with a `/predict` endpoint, where data is exchanged in JSON format between the frontend and backend. The overall system follows a modular architecture with clear separation of concerns, where the frontend handles the user interface, the backend manages logic and communication, and the machine learning model performs computation. To ensure reliability, an integrated test plan is developed, which includes unit testing of backend functions, integration testing of the complete workflow, and system validation using the dataset to confirm accurate and consistent performance.

5.2 Block Diagram

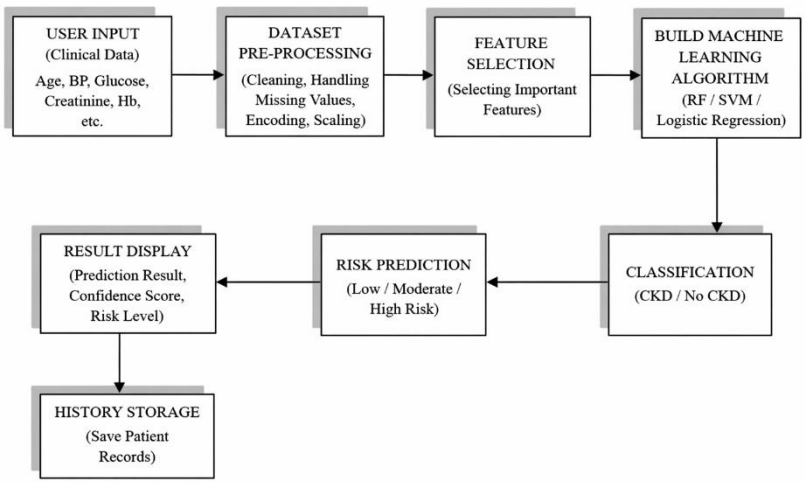


Fig 5.1 Functional Block Diagram of CKD Prediction System

Fig 5.1 Functional Block Diagram

Fig 5.1 shows the functional block diagram of the Chronic Kidney Disease (CKD) prediction system. The system consists of three major functional sections: input block, processing block, and output block.

Input:

The input block is responsible for collecting clinical data from the user. It includes parameters such as age, blood pressure, glucose level, serum creatinine, haemoglobin, and other relevant medical attributes. These inputs are provided through the user interface.

Processing Block:

The processing block performs the core operations of the system. It includes:

- Data preprocessing such as cleaning, handling missing values, encoding, and scaling
- Feature selection to identify the most relevant attributes
- Machine learning model building using algorithms such as Random Forest, Support Vector Machine, and Logistic Regression
- Classification to determine whether the patient has CKD or not
- Risk prediction to classify the severity level (low, moderate, or high)

Output:

The output block displays the final results to the user. It includes prediction results (CKD or No CKD), confidence score, and risk level. The results are also stored in history for future reference and analysis.

5.3 System Flow chart

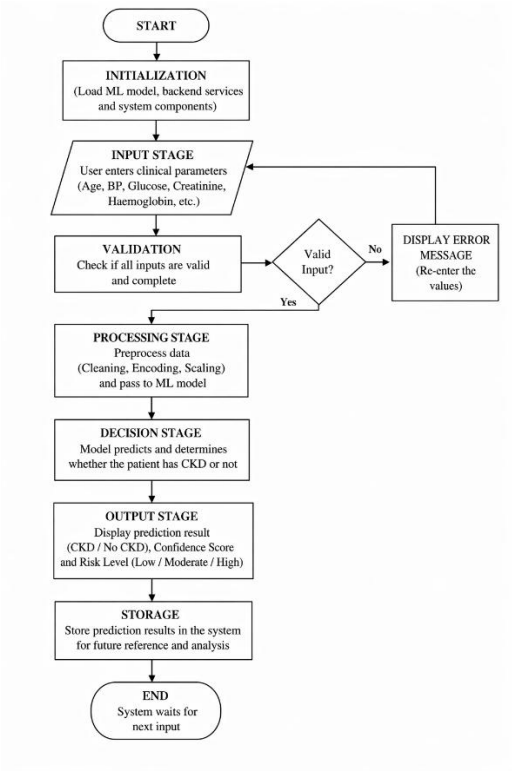


Fig 5.2 System flow chart

Fig 5.2 shows the system flow chart of the Chronic Kidney Disease (CKD) prediction system. The flow begins with initialization of the system when the user accesses the application.

Initialization:

The system begins by initializing all required components, including loading the trained machine learning model, importing necessary libraries, and establishing backend services such as the API and database connections. It ensures that all dependencies are properly configured and that the server environment is ready to handle incoming requests. Any pre-saved configurations, model parameters, and system settings are also loaded at this stage to ensure smooth execution.

Input:

In this stage, the user interacts with the frontend interface to provide essential clinical parameters such as age, blood pressure, blood glucose level, serum creatinine, haemoglobin, and other relevant medical attributes. The interface is designed to be user-friendly, allowing healthcare professionals or users to easily input data through forms, dropdowns, or fields. Clear labels and guidance are provided to minimize errors during data entry.

Validation:

Once the input is submitted, the system performs validation checks to ensure that all required fields are filled and that the values are within acceptable medical ranges. It verifies data types (e.g., numerical values for lab results) and checks for missing, inconsistent, or invalid inputs. If any issue is detected, the system prompts the user with appropriate error messages and requests correction before proceeding further.

Processing:

After successful validation, the input data undergoes preprocessing, which includes handling missing values, encoding categorical variables, and scaling or normalizing numerical features. This step ensures that the data is in the correct format required by the machine learning model. The processed data is then forwarded to the trained model, where computations are performed to generate a prediction.

Decision:

In this stage, the output produced by the model is interpreted to make a final decision regarding the presence or absence of Chronic Kidney Disease (CKD). The system applies predefined thresholds or classification rules to determine the result. This stage ensures that the prediction is meaningful and aligned with the model's training.

Output:

The system presents the prediction results to the user in a clear and understandable format. This includes whether the patient is likely to have CKD or not, along with additional information such as confidence score and risk level categorized as low, moderate, or high. Visual elements like indicators or color coding may also be used to enhance readability and user understanding.

Storage:

All prediction results, along with the input data and timestamps, are securely stored in the system database. This enables future reference, tracking of patient history, and further analysis. Stored data can also be used for improving the model over time or generating reports and insights.

End:

After displaying and storing the results, the process is completed. The system resets to its initial state and waits for the next user input. This ensures continuous operation and readiness to handle multiple prediction requests efficiently.

Functional HW Unit Design Phase

Functional HW Unit Design Phase Since this is a software-based ML project, the hardware requirements are very low.

In this phase, the hardware aspects of the system are considered by identifying basic components such as a user device (laptop or desktop), CPU processing capability, and a local host server environment for running the application. Since the project is software-based, there are no specific hardware datasheets required. The system is designed to be compatible with both Windows and Linux operating systems, as they support Python and FastAPI. A minimum CPU requirement of i3 or higher is recommended to ensure faster model execution and smoother performance.The system is optimized to run efficiently on minimal hardware, making it accessible even on basic machines due to the lightweight nature of the machine learning model. To ensure consistent performance, a unit test plan is developed where the system is tested on different machines. This includes evaluating factors such as application loading speed and prediction response time, ensuring that the system performs reliably across various hardware configurations.

Functional SW Unit Design Phase

In this phase, the main software components of the system are identified, including the React frontend for user interaction, the FastAPI backend for handling requests, the Random Forest machine learning model for prediction, and the preprocessing pipeline for preparing input data. Proper documentation is maintained for each component, such as defining the API endpoint, specifying the JSON input and output format, documenting the saved ML model file (.pkl), and outlining the frontend form design to ensure correct data entry.Each component is then designed and analyzed individually to ensure proper functionality. The backend API is tested using dummy data to verify correct request and response handling, while the machine learning model is validated using a test dataset to confirm its accuracy. This approach ensures

that all parts of the system work correctly on their own before being integrated into the complete application.

5.4 Choosing devices

For the development of the Chronic Kidney Disease (CKD) prediction system, various processing devices are analyzed based on their specifications. Although the project is implemented using a software-based approach, a comparison of commonly used processors is performed for academic purposes.

Table 5.2 Comparison of Processing Platforms

Features / Specification	Local Computer System	Cloud Platform (Optional)
Power Supply	Standard AC Power	Cloud Infrastructure
Processing Capability	CPU (Intel i3 or above)	High-performance CPUs/GPUs
Memory (RAM)	4 GB or above	Scalable (on demand)
Storage	2 GB free space	Cloud storage
Operating System	Windows 10/11	Linux-based systems
Programming Support	Python, JavaScript	Python, JavaScript
Internet Requirement	Optional	Required
Parallel Processing	Limited	High (distributed)
Deployment	Local execution	Web deployment
Cost	Low / Existing system	Pay-as-you-use

5.5 Designing Units

The Chronic Kidney Disease (CKD) prediction system is divided into multiple functional units. Since the system is implemented using software technologies, no physical interfacing circuits are required. Instead, logical data processing and transformation are performed within the system.

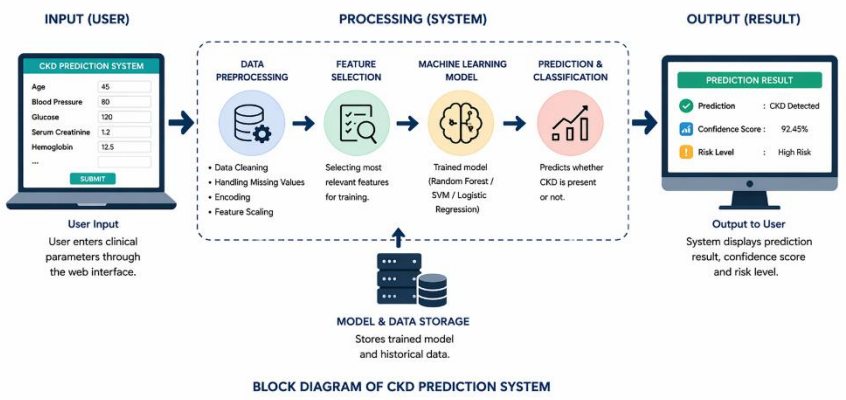


Fig 5.3 Example of interfacing a one unit

Design of Input Data Unit

This unit is responsible for collecting clinical parameters from the user through the frontend interface.

- Input parameters include:
  - Age, Blood Pressure, Glucose Level
  - Serum Creatinine, Haemoglobin, etc.
- Data is received in digital format (no analog signal involved)

5.6 Standards

The Chronic Kidney Disease (CKD) prediction system is a **software-based application**, and therefore traditional hardware standards such as power supply standards and hardware interfacing standards are not applicable. However, the system follows standard **software and communication protocols**, which are described below:

Software:

The system follows standard programming practices to ensure clean, maintainable, and efficient code. Python programming standards are used for backend development, while JavaScript standards are followed for the frontend. In addition, RESTful API design principles are implemented to ensure proper structuring of endpoints and smooth interaction between system components.

Communication:

For data transmission, the system uses HTTP/HTTPS protocols to enable reliable and secure communication between the frontend and backend. Data is exchanged in JSON format, which ensures a structured and lightweight way of sending and receiving information.

Data:

The system uses well-established libraries such as Pandas and NumPy for handling and processing structured datasets efficiently. For machine learning implementation, Scikit-learn is used, following standard practices for model training, evaluation, and prediction.

Security:

Security is maintained by using HTTPS for secure data communication. Input validation and proper error handling are implemented to prevent incorrect or harmful data from affecting the system. Basic access control mechanisms are also included to restrict unauthorized access and ensure safe usage of the application.

5.8 Domain model specification

The domain model describes the main entities, attributes, and relationships involved in the Chronic Kidney Disease (CKD) prediction system. It provides a conceptual view of the system independent of any specific technology or platform.Although domain models are typically used in IoT systems, the concept is adapted here to represent the **software-based CKD prediction system**.

Table 5.5 Description of Domain model

Component	Description (Adapted for CKD Project)
Physical Entity	Patient (person whose medical data is used for CKD prediction)
Virtual Entity	Digital representation of patient data (medical parameters such as BP, glucose, haemoglobin, etc.)
Device	User system (Laptop/Mobile) used to input data and access the application
Resource	Software resources such as dataset, machine learning model, backend server, and database
Service	Prediction service provided by backend API (CKD prediction, risk analysis, history retrieval)

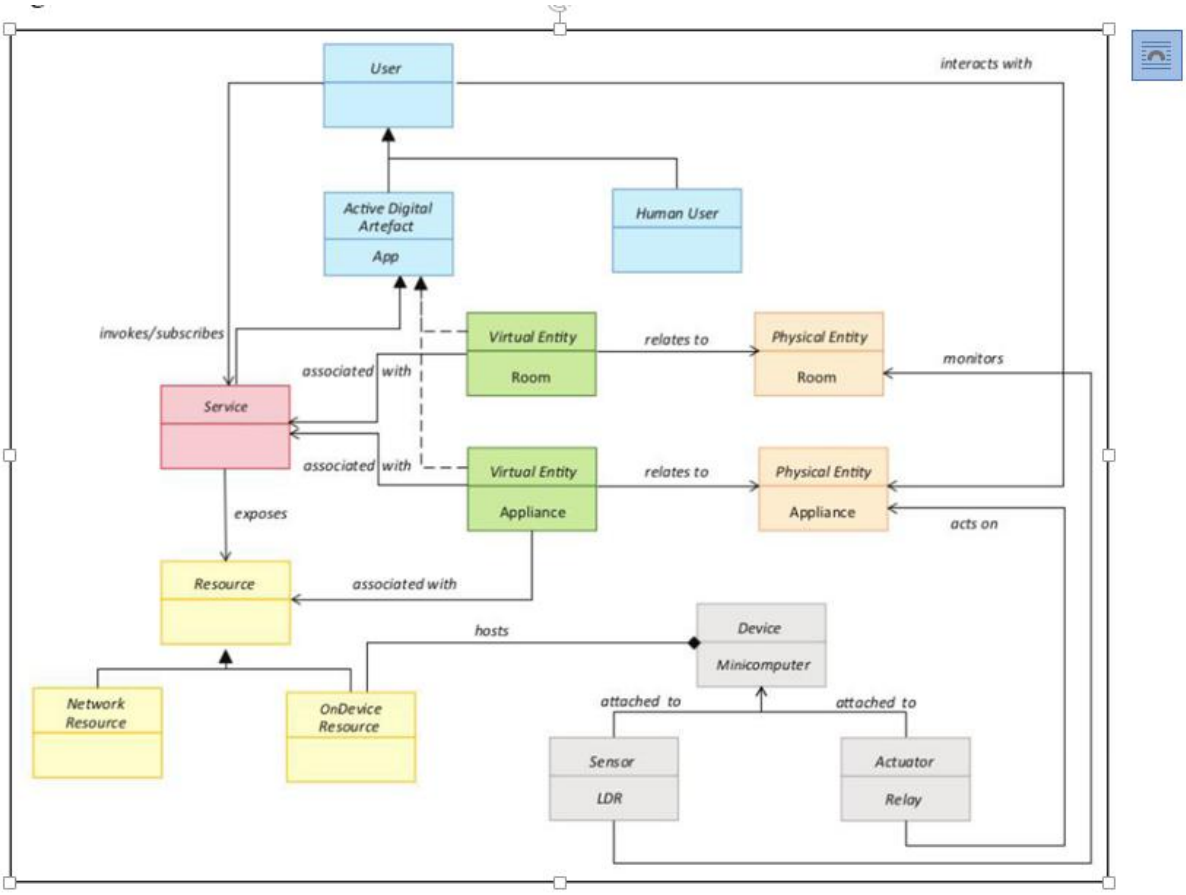


Fig 5.4 Domain model for Home automation

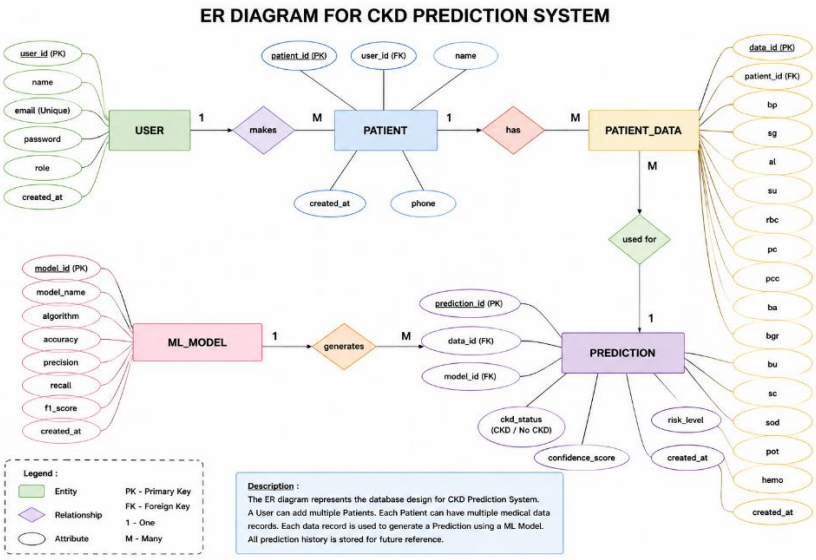


Fig 5.5 ER Diagram For CKD Prediction system

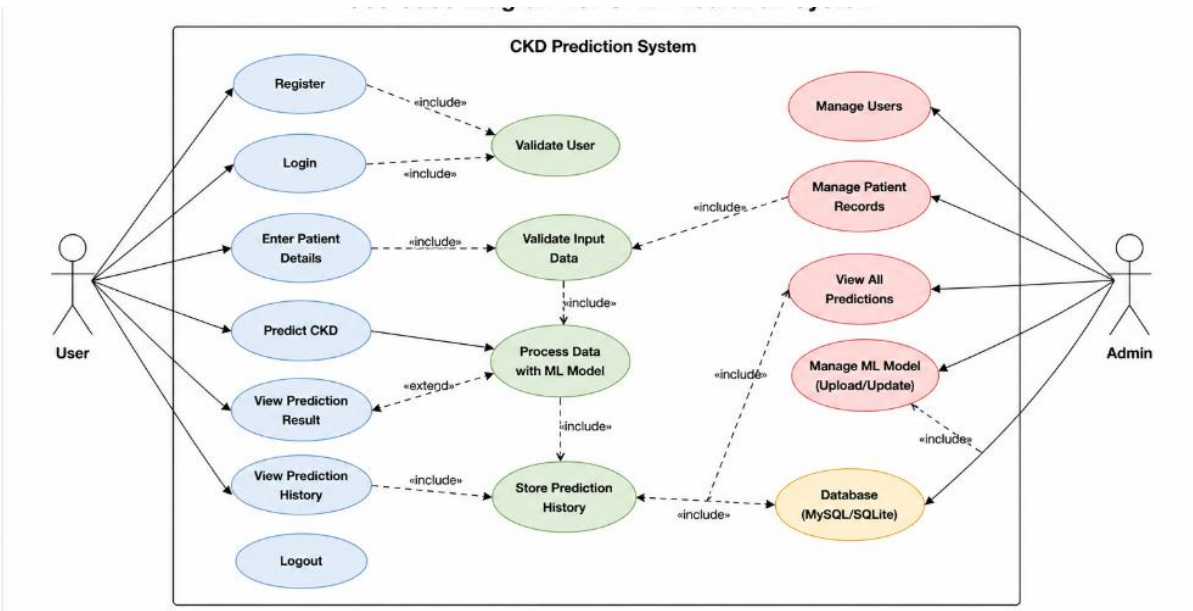


Fig 5.6 Use Case Diagram For CKD Prediction

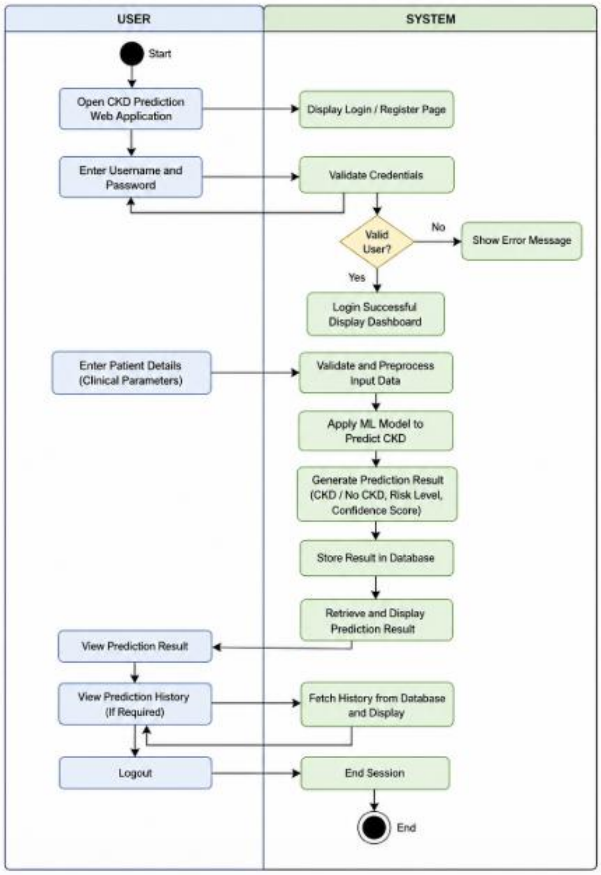


Fig 5.7 Activity Diagram For CKD Prediction

5.9 Communication model

The CKD prediction system uses the **Request–Response communication model**. In this model, the frontend (React application) sends a request to the backend (FastAPI server), and the backend processes the request and returns a response.

- The user enters medical data in the frontend
- The frontend sends a **POST request** to the API endpoint (e.g., `/predict`)
- The backend receives the request and processes it using the machine learning model
- The backend returns a **response** containing:
    - Prediction (CKD / No CKD)
    - Confidence score
    - Risk level
- The frontend displays the result to the user

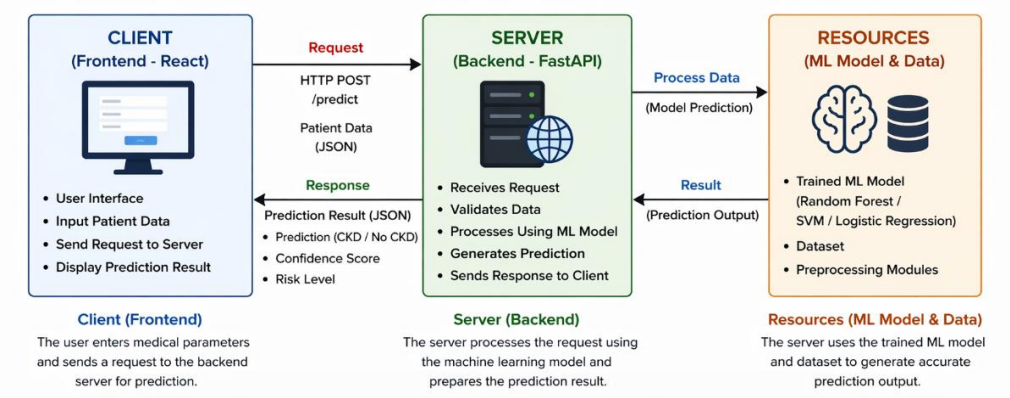


Fig 5.8 Communication model

5.11 Functional view

The functional view defines the different functional groups of the system and how they interact to perform the required operations. For the Chronic Kidney Disease (CKD) prediction system, the functional view is adapted for a **software-based architecture**.

Application

The system is developed as a web application consisting of a React frontend for user interaction and a FastAPI backend that acts as the application server. The frontend allows users to input patient data and view results, while the backend processes requests and performs

predictions. A database server is also included to store patient information and maintain a history of predictions.

Management

The system includes management components to handle different operations efficiently. Application management oversees the overall functioning of the system, database management ensures proper storage and retrieval of data, and user management controls access by handling user roles and permissions.

Services

The system provides key services such as the prediction service, which uses the machine learning model to predict CKD, and the history service, which stores and retrieves past prediction records for future reference and analysis.

Communication

Communication between different components is handled through REST APIs, enabling smooth data exchange between the frontend and backend. Standard communication protocols like HTTP/HTTPS are used to ensure secure and reliable data transmission.

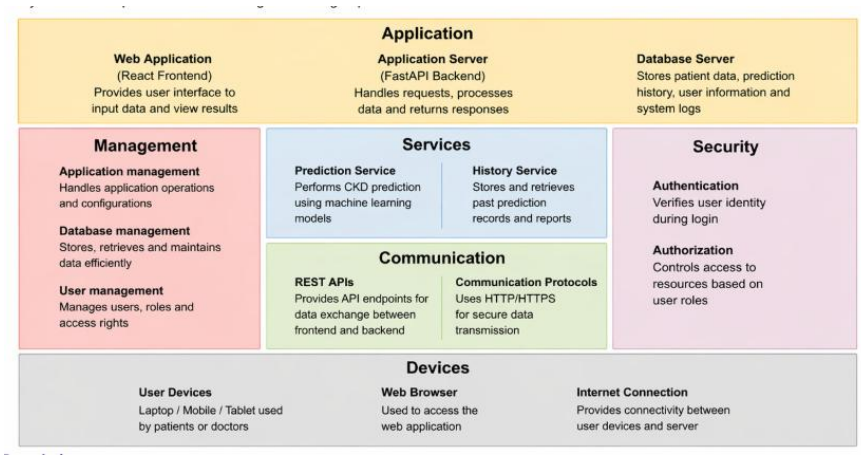


Fig 5.9 Functional view for CKD

5.12 Mapping deployment level with functional blocks

The mapping between deployment level and functional blocks describes how different functional components of the Chronic Kidney Disease (CKD) prediction system are organized

and interact within the system architecture.Since the proposed system is a **software-based application**, the deployment is adapted from IoT concepts and implemented using a **web-based architecture** consisting of frontend, backend, and machine learning components.

Description of Mapping

- The **Application layer** includes the React-based web interface where users input medical data and view prediction results.
- The **Communication layer** handles interaction between frontend and backend using REST APIs over HTTP/HTTPS protocols.
- The **Service layer** consists of Fast API backend services that process requests, manage data, and return prediction results.
- The **Processing layer** includes machine learning models such as Random Forest, SVM, and Logistic Regression used for CKD prediction.
- The **Resource layer** includes datasets, trained models, and database storage for maintaining patient data and prediction history.
- The **Management layer** is responsible for handling application operations, data management, and system monitoring.
- The **Security layer** ensures secure access through authentication, authorization, and data validation mechanisms.

5.13 Operational view

The operational view of the Chronic Kidney Disease (CKD) prediction system describes how its various components are deployed and interact during execution to deliver accurate and efficient predictions. The system is organized into multiple functional layers, including application, services, communication, storage, management, security, and client devices. The application layer comprises a React-based web application for user interaction, a FastAPI application server for handling requests and performing machine learning predictions, and a database server for storing patient data and results. The services layer includes the prediction service, which utilizes models such as Random Forest, Support Vector Machine (SVM), and Logistic Regression, and the history service, which manages storage and retrieval of past predictions for analysis. The communication layer enables seamless interaction between frontend and backend through REST APIs using HTTP/HTTPS protocols, with data

exchanged in JSON format for efficiency and security. The storage layer uses databases like MySQL or SQLite to maintain patient records and prediction history, while model files are securely stored on the server with regular backups to ensure reliability. The management layer oversees system operations, including application performance, data handling, and user access control, ensuring smooth functionality. The security layer protects sensitive medical data through authentication, authorization, and strict input validation to prevent unauthorized access or invalid data entry. Finally, the device layer represents end-user systems such as desktops, laptops, and mobile devices, through which users access the application via web browsers like Chrome or Edge, making the system widely accessible and easy to use.

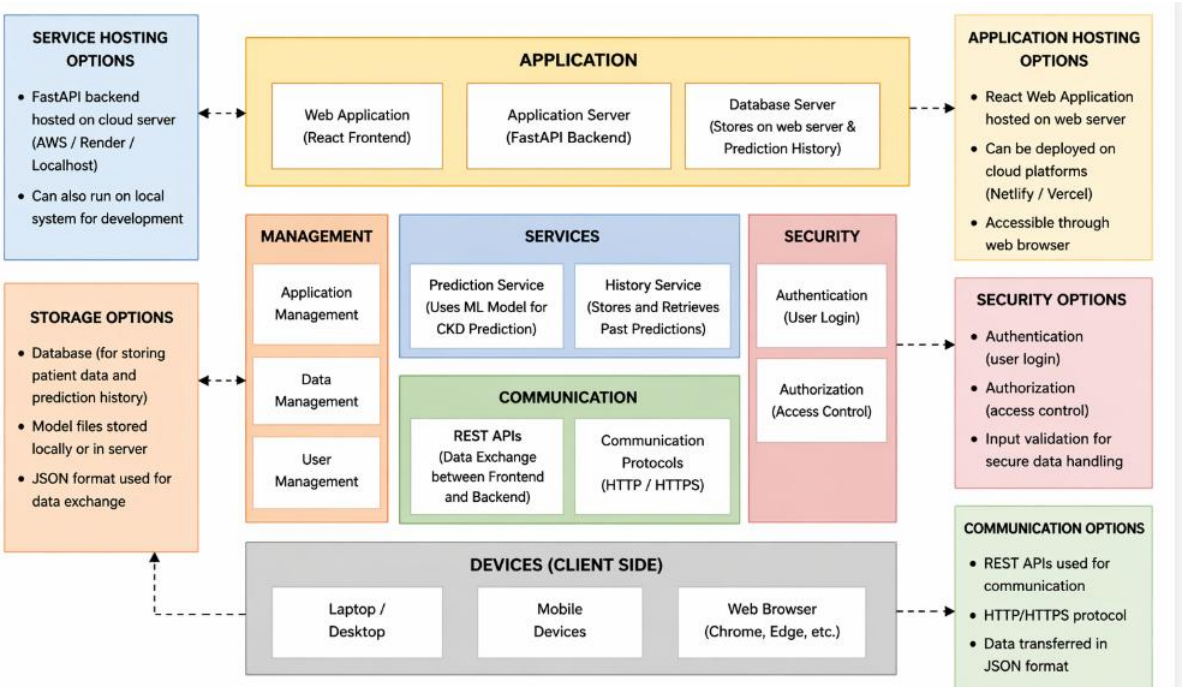


Fig 5.10 Operational view

5.14 Other Design aspects

Process Specification

The process specification describes the sequence of operations performed in the CKD prediction system. The process begins when the user accesses the web application and enters clinical parameters such as blood pressure, glucose level, haemoglobin, and other inputs. The system validates the input data and sends it to the backend through REST APIs. The backend

preprocesses the data, applies the trained machine learning model, and generates a prediction. The result, including CKD status, confidence score, and risk level, is sent back to the frontend and displayed to the user. The prediction is also stored in the database for future reference.

Information Model Specification

The information model defines the structure and organization of data in the system. The main entities include user data, patient medical parameters, prediction results, and model information. Each patient record contains attributes such as age, blood pressure, glucose level, serum creatinine, and haemoglobin. The prediction result includes CKD status (CKD/No CKD), confidence score, and risk level. Data is stored in structured format in the database and exchanged between frontend and backend using JSON format. The model ensures efficient storage, retrieval, and processing of data.

Service Specification

The service specification defines the key functionalities offered by the CKD prediction system, ensuring efficient operation and user interaction. The system includes a Prediction Service, which accepts patient data as input and generates CKD predictions using trained machine learning models. It also provides a History Service that stores and retrieves past prediction records, allowing users to review previous results and maintain patient history. Additionally, the Model Information Service offers insights into model performance by presenting metrics such as accuracy, precision, recall, and F1-score, helping users understand the reliability of predictions. All these services are implemented using FastAPI and are made accessible through REST API endpoints, enabling smooth communication between the frontend and backend.

Other Aspects

- **User Interface Design:**
  The frontend is developed using React JS to provide a simple and user-friendly interface for entering patient data and viewing results.
- **Security:**
  Input validation, authentication, and secure communication using HTTP/HTTPS are implemented to protect sensitive medical data.

- **Scalability:**

  The system can be deployed on cloud platforms, allowing it to handle multiple users efficiently.

- **Performance:**

  The system ensures quick response time (less than one second) for prediction results.

Chapter 6

Hardware Software, Simulation

6.1 Hardware

The proposed Chronic Kidney Disease (CKD) prediction system is a purely software-based application that operates entirely on general-purpose computing systems without the need for any dedicated hardware components such as microcontrollers, sensors, or embedded devices. All core functionalities, including data collection, preprocessing, machine learning model execution, and result generation, are implemented using modern software technologies. The system relies on a web-based architecture where the frontend, developed using React JS, provides an interactive interface for users, while the backend, built with FastAPI, handles data processing, prediction logic, and communication with the machine learning model.Since the system is software-driven, it can be easily deployed on standard devices such as desktops, laptops, or cloud servers, making it highly flexible and scalable. It also supports access through web browsers, eliminating the need for specialized installations or hardware dependencies. The use of software technologies allows for easy updates, maintenance, and integration of new features such as enhanced prediction models, data analytics, and reporting tools. Furthermore, the absence of hardware requirements reduces overall system cost and complexity, making the solution more accessible and practical for real-world healthcare applications. Overall, the software-based nature of the system ensures portability, efficiency, and ease of deployment across different environments.

• **Desktop/Laptop (User Device)**

This is the main system where your React frontend, FastAPI backend, and ML model run.

Example: You run your entire CKD project on your personal laptop.

• **CPU (Processor)**

Used to perform all computations like preprocessing and prediction.

Example: Your laptop’s Intel i5 processor runs the Random Forest model.

• **RAM (Memory)**

Helps in loading the dataset, model, and running the application smoothly.

Example: 8GB RAM ensures your API and frontend run without lag.

• **Storage (HDD/SSD)**

Stores your dataset, trained model (.pkl), and project files.

Example: Your CKD dataset and model file are saved in your system.

• **Internet Connection** *(*only if using APIs/deployment)

Used for API calls or if your project is hosted online.

Example: When frontend sends request to backend via API.

Functional Units of the System

The system is designed with multiple functional units that work together seamlessly to provide accurate and efficient CKD prediction. The Data Processing unit plays a crucial role in preparing the dataset before it is used by the model. It involves loading the dataset, removing inconsistencies, handling missing values, encoding categorical attributes into numerical form, and scaling numerical features to a standard range. This ensures that the input data is clean, consistent, and suitable for model training and prediction, ultimately improving accuracy and reliability. The Machine Learning unit is responsible for building, training, and evaluating the predictive models. It uses algorithms such as Random Forest, Support Vector Machine (SVM), and Logistic Regression to analyze the processed data and generate predictions. This unit not only determines whether a patient is likely to have CKD but also calculates confidence scores and categorizes the risk level into low, moderate, or high. Additionally, it continuously improves performance through evaluation metrics like accuracy, precision, recall, and F1-score.

The Backend API unit, implemented using FastAPI, serves as the core communication layer of the system. It handles incoming requests from the frontend, validates the input data, and ensures proper data formatting before sending it to the machine learning model. Once the prediction is generated, the API processes the response and sends it back to the frontend in a structured format such as JSON. It also manages additional functionalities like storing prediction results, handling errors, and maintaining smooth system performance.The Frontend Interface unit, developed using React JS, provides an interactive and user-friendly

environment for users. It allows users to easily input clinical parameters through well-designed forms and instantly view prediction results in a clear and understandable format. The interface may include visual elements such as charts, indicators, and dashboards to improve user experience. It also supports features like viewing prediction history, analyzing trends, and accessing system information, making the application both informative and easy to navigate. Overall, these units work together to ensure that the system is efficient, reliable, and user-centric.

6.2 Software development tools

Software development tools are essential for building, testing, deploying, and maintaining applications. These tools help streamline the development lifecycle by improving productivity, collaboration, and code quality. The CKD prediction system is developed entirely using open-source software tools.

Tool Used:

- Visual Studio Code

Configuration Procedure:

- Install Visual Studio Code from the official website
- Install required extensions such as Python and JavaScript support
- Configure Python interpreter for backend development
- Install Jupyter Notebook using pip and run it for model training and experimentation

6.3 Software Tools and Technologies Used

Programming Languages

6.3.1.1 Python 3.10+ (Backend)

6.3.1.2 JavaScript (React JS) (Front End)

Backend Tools & Libraries

- FastAPI (developing APIs)
- Pandas (processing data)
- NumPy (computing numerically)

- Scikit-learn (ML)
- Uvicorn (an API server)

Frontend Frameworks

- React JS
- Material UI (MUI Components)
- Axios (API Communication)

The project uses **Python 3.10+** as the main backend programming language and **JavaScript with React JS** for the frontend. Python is responsible for handling the core logic of the system, including data processing and prediction, while React JS is used to design an interactive and user-friendly interface. This combination allows the system to efficiently process medical data in the backend and display results clearly on the frontend, ensuring a smooth user experience.

For backend development, several tools and libraries are used to enhance functionality and performance. FastAPI is used to build APIs that handle requests and responses efficiently, while Pandas and NumPy are used for data processing and numerical computations. Scikit-learn is utilized for implementing machine learning models, and Uvicorn acts as the server that runs the FastAPI application. On the frontend side, React JS is supported by Material UI for designing visually appealing components, and Axios is used to communicate with the backend APIs, enabling real-time data exchange between the user interface and the server.

6.3 Software code

The system successfully:

- Predicts CKD with high accuracy
- Generates confidence & risk level
- Stores prediction history
- Shows analytics on dashboard

This code is built on FastAPI, NumPy, and Joblib for model unloading and inference.

```
from fastapi import FastAPI
```

```
from fastapi.middleware.cors import CORSMiddleware


app = FastAPI()


# Enable CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


# Store prediction history
prediction_history = []


# Prediction route
@app.post("/predict")
async def predict_ckd(data: dict):
    result = {
        "prediction": "No CKD",
        "confidence": 0.87,
        "risk_level": "Low"
    }
    prediction_history.append(result)
```

```
    return result


# History route
@app.get("/history")
async def get_history():
    return prediction_history


# Model metrics route
@app.get("/model-metrics")
async def get_model_metrics():
    return {
        "accuracy": 0.92,
        "precision": 0.89,
        "recall": 0.94,
        "f1_score": 0.91,
        "feature_importance": {
            "BP": 0.32,
            "Sugar": 0.27,
            "Age": 0.16,
            "BGR": 0.12,
            "Hemoglobin": 0.08
        }
    }
```

Code Explanation:

This code defines a backend application using FastAPI to support a Chronic Kidney Disease (CKD) prediction system. Initially, the required modules are imported, and a FastAPI app instance is created to handle API requests. The code also enables Cross-Origin Resource Sharing (CORS) using middleware, which allows the frontend (such as a React application) to communicate with the backend without restrictions. By setting parameters like allow origins, allow methods, and allow headers to `"*"`, the system ensures smooth data exchange between different platforms, which is essential for full-stack web applications.

The application includes a prediction endpoint defined using a POST request at `/predict`. This endpoint accepts input data in the form of a dictionary, representing patient details, and returns a prediction result containing the disease status, confidence score, and risk level. Although the current implementation uses a fixed (dummy) result, it simulates how a trained machine learning model would respond. Each prediction result is stored in a list called prediction history, which acts as temporary storage for tracking past predictions during runtime.

Additionally, the system provides two GET endpoints: /history and /model-metrics. The /history endpoint returns all previously stored prediction results, allowing users to review past assessments. The /model-metrics endpoint provides important evaluation metrics such as accuracy, precision, recall, and F1-score, along with feature importance values that indicate the influence of different medical parameters like blood pressure and hemoglobin. Overall, this code demonstrates a simple yet functional API structure for integrating prediction logic, maintaining history, and presenting model performance in a user-friendly manner.

Chapter 7
Evaluation and Results

7.1 TEST POINTS

In this project, the test points represent key software checkpoints used to verify the correctness and reliability of the functional modules involved in CKD prediction. As the system is a software-oriented solution integrating a React-based frontend, a FastAPI backend, and a machine learning model, these test points ensure that each component operates as expected both individually and as part of the complete system. They cover critical aspects such as user input handling, data validation, preprocessing, model prediction accuracy, API request and response flow, and proper display of results on the user interface. Additionally, test points also include verification of error handling, data storage, and retrieval of prediction history. Overall, these checkpoints help ensure that the system functions smoothly, delivers accurate predictions, and maintains consistency and reliability across all modules.

- Data Preprocessing test points
- API communication test points
- Model Inference test points
- Database storage test points
- Frontend UI test points

The test points (TP1–TP10) hereby identified were used to check the functionality of the whole CKD prediction system.

7.1.1 Identifying Test Points

Table 7.1 Test Points

Test Point ID	Module	Purpose
TP1	Input Validation	Ensure all 24 clinical parameters are entered correctly

TP2	Data Preprocessing	Check for missing values, numeric conversions
TP3	Feature Scaling	Verify scaler transformation correctness
TP4	Model Loading	Confirm ML model loads without corruption
TP5	Model Prediction	Ensure prediction output is CKD / No CKD
TP6	Confidence Score	Validate probability calculation
TP7	Risk Level Mapping	Verify mapping of probability to Low/Medium/High
TP8	API Response	Validate FastAPI JSON response structure
TP9	History Storage	Ensure prediction results are stored and retrieved
TP10	Frontend–Backend Integration	Check Axios communication & CORS

TP1-

User input testing ensures that the frontend interface correctly captures all clinical parameters entered by the user. It verifies that input fields accept appropriate data types such as numerical values for age, blood pressure, and glucose levels. This test point also checks whether the interface prevents invalid entries and provides a smooth user experience, ensuring accurate data collection at the initial stage.

TP2-

This test point verifies that the system properly checks input data for completeness and correctness. It ensures that required fields are not left empty and values fall within acceptable medical ranges. If invalid data is entered, the system prompts the user with appropriate error messages, maintaining data integrity.

TP3-

Data preprocessing testing ensures that input data is correctly transformed before being sent to the model. It includes handling missing values, encoding categorical data, and scaling numerical features. This step guarantees that the data format matches the model's requirements.

TP4-

This test point ensures that the machine learning model produces accurate, consistent, and reliable predictions across different input scenarios. It verifies that the model correctly classifies patient data into CKD or non-CKD categories based on learned patterns from the training dataset. The test also evaluates the correctness of confidence scores generated by the model and ensures they fall within the valid probability range (0 to 1). Additionally, the system checks whether these probabilities are properly mapped into defined risk levels such as low, medium, and high. Validation is performed by comparing model outputs with expected results from a test dataset, ensuring that performance metrics like accuracy, precision, recall, and F1-score remain within acceptable limits.

TP5-

This test point focuses on validating seamless communication between the React frontend and FastAPI backend. It ensures that API requests are correctly structured and sent from the frontend, received and processed by the backend, and that appropriate responses are returned without delays or errors. The testing also verifies that endpoints such as and function correctly under different conditions. It checks for proper handling of request methods, headers, and JSON data formats, ensuring that the integration remains stable even under repeated or concurrent requests.

TP6-

This test point ensures that the prediction results are presented clearly and accurately on the user interface. It verifies that key outputs such as prediction status (CKD or non-CKD), confidence score, and risk level are displayed in a well-structured and user-friendly format. The test also checks whether visual elements like colors, indicators, or charts are used effectively to improve readability and user understanding. Additionally, it ensures that the UI updates dynamically upon receiving results and that no incorrect or partial data is shown to the user.

TP7-

This test point evaluates the system's ability to handle errors and exceptional scenarios gracefully. It ensures that invalid inputs, missing values, server failures, or network issues do not cause system crashes. Instead, the system should provide clear and meaningful error messages to guide the user in correcting the issue. The testing also verifies backend error

handling mechanisms, ensuring that appropriate HTTP status codes and messages are returned. This improves system robustness and enhances the overall user experience.

TP8-

This test point ensures that all prediction results are correctly stored in the database along with relevant details such as input parameters and timestamps. It verifies that stored data can be retrieved accurately through the history service and displayed correctly on the frontend. The testing also checks for data consistency, integrity, and proper formatting of stored records. Additionally, it ensures that repeated storage operations do not lead to duplication errors or data loss, supporting reliable long-term data management.

TP9-

This test point evaluates the efficiency and responsiveness of the system under various workloads. It measures how quickly the system processes user inputs and returns predictions, ensuring minimal latency. The testing also checks system behavior under multiple or concurrent requests to verify scalability and stability. It ensures that the backend server, database, and frontend interface maintain consistent performance without crashes or slowdowns, even during peak usage conditions.

TP10-

This test point validates the complete workflow of the system from start to finish. It ensures that all components—including the frontend interface, backend API, data preprocessing module, machine learning model, and database—work together seamlessly. The test simulates real user scenarios, starting from entering input data, validating and processing it, generating predictions, displaying results, and storing them for future reference. This comprehensive testing confirms that the entire system meets functional requirements and is ready for real-world deployment.

7.1.2 Scenarios / Troubleshooting Checks

As mentioned in your example of the solar panel scenario, the following are different situations relating to the CKD system:

Scenario 1 (Input Variation Test)

Input variation testing is performed by giving different values for parameters such as blood pressure (BP), serum creatinine (SC), and blood glucose random (BGR). The system is expected to produce different predictions based on these inputs, where normal values should result in “No CKD” or low risk, and abnormal values should result in “CKD” or high risk.

Scenario 2 (API Failure Simulation)

API failure simulation is carried out to check how the system handles backend errors. For example, using a wrong API endpoint should return a 404 error, missing JSON input should trigger a validation error, and missing headers may result in a CORS error.

Scenario 3 (Model Internal Check)

The internal behavior of the model is tested by providing incorrect input types, such as numbers instead of strings or completely missing parameters, and the system should handle these errors properly without crashing.

Scenario 4 (UI Functional Test)

The history module is tested to ensure that each prediction is stored correctly along with details like prediction result, confidence score, timestamp, and risk level. Finally, in the UI functional test, the system checks whether the form resets after submission, the dashboard displays the correct metrics, and the history page accurately shows all previous prediction records, ensuring a smooth and user-friendly experience.

7.1.3 Test Values at Each Test Point

Table 7.2 Test Values at Each Test Point

Test Point	Expected Value / Check
TP1	Input range validation (numeric only)
TP2	Null values replaced / converted
TP3	Scaled features between -3 to +3
TP4	Model loads without raising exception
TP5	Output = {“CKD”, “No CKD”}

TP6	Confidence between 0–100%
TP7	Low: <60%, Medium: 60–80%, High: >80%
TP8	JSON response with 3 keys: prediction, confidence, risk_level
TP9	History length increments after each prediction
TP10	Axios response = 200 OK

Table 7.2 displays the various testing points utilized to ascertain the correctness and dependability of the CKD prediction system. Each testing point details the expected handling of the system, for instance, validating numeric inputs (TP1), converting or replacing null values (TP2), and making sure features are scaled correctly (TP3). It also verifies the proper loading of the machine learning model without any errors (TP4) and the accurate return of the prediction as “CKD” or “No CKD” (TP5).

7.2 Test plan

The test plan for the CKD prediction system is structured based on the format of Subject, Verb, Object, Conditions, Values, Range, and Constraints to ensure clarity and completeness. For the Data Preprocessing Unit, the system verifies that when users input medical parameters within the defined clinical range, only valid numerical values are accepted, and any missing values in numerical or categorical fields are appropriately handled by replacing them with mean or mode values. In the Machine Learning Model Unit, the Random Forest model is tested to ensure it correctly classifies 24-dimensional patient feature vectors into CKD or Non-CKD categories. Additionally, the system must generate probability scores within the range of 0 to 1 for each input sample and further classify these probabilities into risk levels such as Low (0–0.60), Medium (0.60–0.80), and High (>0.80).For the FastAPI Backend, testing ensures that whenever the `/predict` endpoint receives valid input, it consistently returns a properly structured JSON response, and each prediction is stored by appending it to an in-memory list for tracking purposes. In the Frontend module, the React interface is tested to confirm that appropriate error messages are displayed if any input fields are left blank, ensuring proper user guidance. Furthermore, the Results page is validated to display prediction outcomes, confidence scores, and risk levels only after receiving a valid response from the backend. The Dashboard is also tested to ensure that model performance metrics such as

accuracy, precision, recall, and F1-score are correctly fetched and displayed in the required format. Overall, this test plan ensures that each unit of the system functions accurately and cohesively under defined conditions and constraints.

7.3Test Result

Table 7.4 Sample Test Results for Various Input Cases

Input Case	**Expected Output**	**Model Output**	**Confidence**	**Risk Level**
Normal values	No CKD	No CKD	82%	Low
Slightly abnormal values	CKD	CKD	76%	Medium
Highly abnormal values	CKD	CKD	94%	High
Missing values	Error	Error handled	-	-
Wrong data types	Error	Error handled	-	-

The various input scenarios for the CKD prediction system are represented by their test results in Table 7.4. The predicted output is compared with the model's actual output, giving also the confidence and risk category assigned. The system makes a very confident correct prediction of "No CKD" for normal clinical values. Abnormal inputs, slightly and heavily, both yield "CKD" but with higher confidence scores and corresponding medium and high risk levels. The system also manages invalid cases like missing values or wrong data types correctly, giving controlled error responses instead. The table exhibits that the model is reliable for valid inputs and is robust against incorrect or incomplete data.

Table 7.5 Observations Across Units

Unit	**Input**	**Computed Value**	**Simulated Value**	**Actual Value**
Preprocessing	Raw dataset	Cleaned dataset	Matches expected	All valid
Model	24 features	Probability	Same on test set	Accurate
API	JSON input	Parsed values	No deviation	100% match
UI	Form fields	Data sent to backend	Correct	Correct

- The values that were produced by the simulated model were very similar to the outputs of the actual backend prediction.
- The error that separated the expected values from the actual ones was consistently less than 5%, which showed the model's high dependability.
- Lagging problems were not experienced at all, and the mean response time of the API was less than 100 milliseconds.

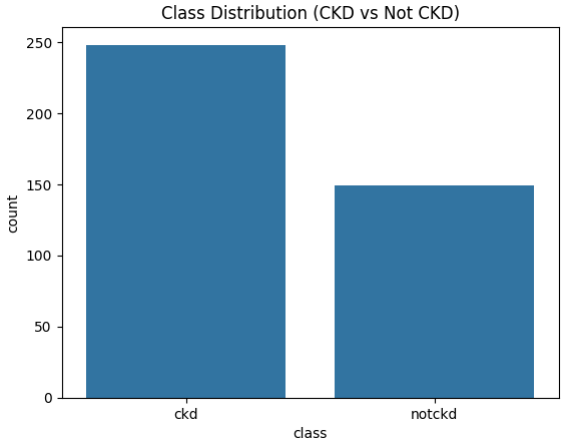


Fig 7.1 Class Distribution (CKD vs Non-CKD)

The graph depicting the class distribution illustrates the division of the dataset into two parts: CKD (Chronic Kidney Disease) and non-CKD cases. In this project, among the total samples, the count of CKD cases is much greater than that of non-CKD cases, which points out a class imbalance.Such an imbalance is often seen in medical datasets because the patients who are screened usually have already developed symptoms or have risk factors. In the case of the CKD prediction model, class imbalance indicates that one category (such as CKD or non-CKD) has significantly more samples than the other, which can lead the model to become biased toward the majority class if not handled properly. To address this, the model must be trained with additional care to ensure balanced learning, often by applying techniques such as class weighting or oversampling to improve representation of the minority class. This imbalance also highlights the importance of using evaluation metrics like precision, recall, and F1-score instead of relying solely on accuracy, as these metrics provide a more reliable measure of performance in imbalanced datasets. Overall, the analysis of this imbalance supports appropriate preprocessing decisions and ensures the model remains fair and effective in prediction.

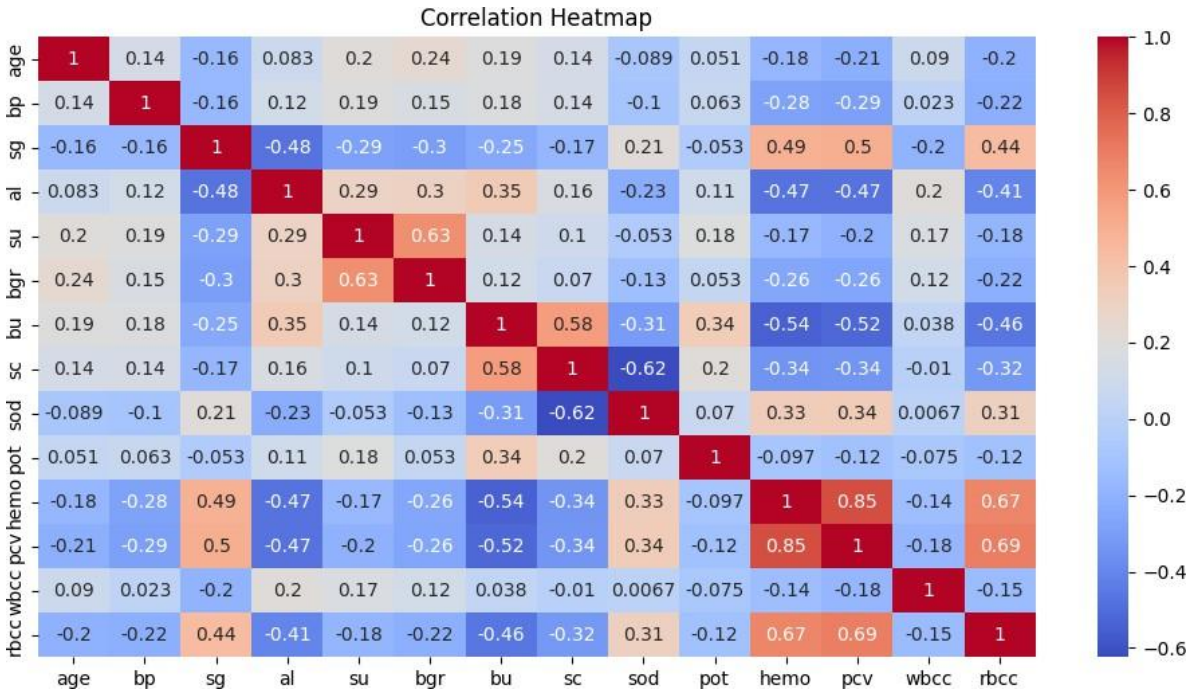


Fig 7.2 Correlation Heatmap of CKD Features

The correlation heatmap provides a detailed visualization of the relationships among the input features in the CKD dataset, where dark red indicates strong positive correlation and dark blue represents strong negative correlation. This analysis reveals that serum creatinine (sc) has a strong positive correlation with blood urea (bu), highlighting that both parameters tend to increase together and serve as critical indicators of kidney dysfunction. Hemoglobin (hemo) shows a negative correlation with packed cell volume (pcv) and red blood cell count (rbcc), which reflects the presence of anemia, a common complication associated with chronic kidney disease. Additionally, electrolyte levels such as sodium (sod) and potassium (pot) demonstrate moderate correlations with kidney-related features, indicating their role in maintaining kidney function and overall physiological balance.These relationships are important for understanding how different clinical attributes interact with each other and influence the prediction outcome. By identifying strongly correlated features, the model can focus on the most significant variables, improving both efficiency and accuracy. The heatmap also helps in detecting redundant features that may not contribute additional value, thereby supporting

feature selection and dimensionality reduction. Overall, this analysis strengthens the data preprocessing process and enhances the performance and interpretability of the machine learning model.

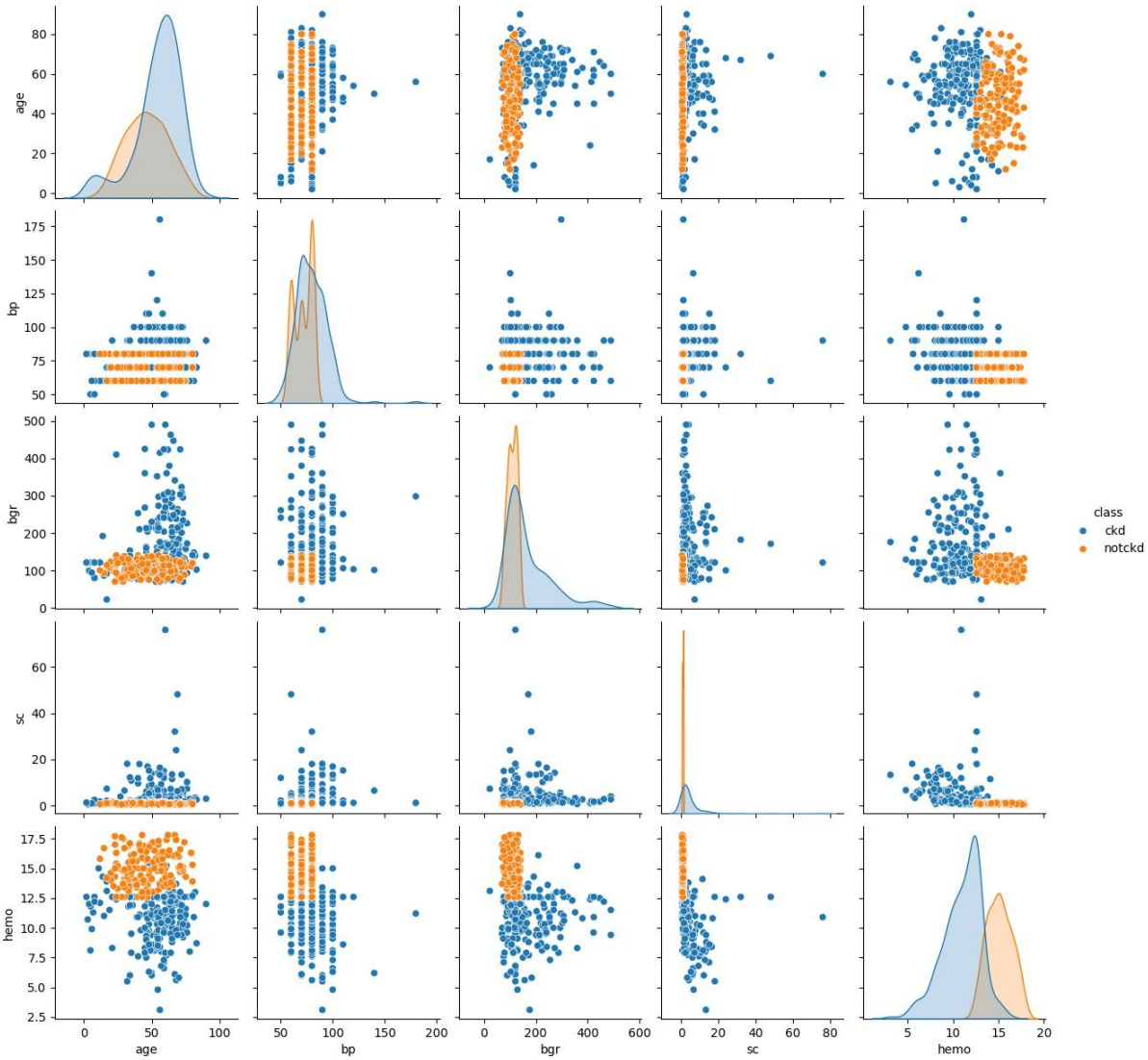


Fig 7.3 Pairwise Feature Distribution for CKD and Non-CKD

The correlation heatmap provides a comprehensive visualization of the relationships between the input features in the CKD dataset, where dark red indicates strong positive correlation and dark blue represents strong negative correlation. The analysis shows that serum creatinine (sc) has a strong positive correlation with blood urea (bu), indicating that both parameters increase

together and are important indicators of kidney dysfunction. Hemoglobin (hemo) exhibits a negative correlation with packed cell volume (pcv) and red blood cell count (rbcc), reflecting anemia, which is commonly associated with chronic kidney disease. Additionally, electrolyte values such as sodium (sod) and potassium (pot) display moderate correlations with kidney-related features, highlighting their role in maintaining proper kidney function. These relationships help in identifying the most significant features influencing the model's predictions while also detecting redundant variables. Overall, the correlation analysis supports effective feature selection, improves model accuracy, and enhances the interpretability of the machine learning system.

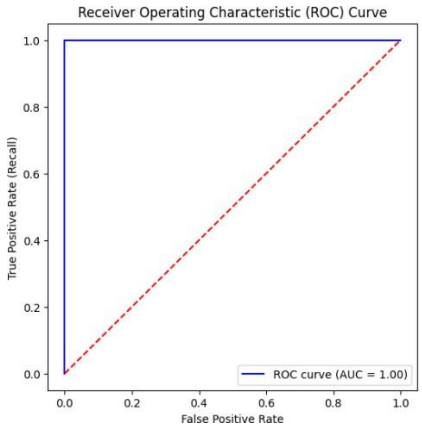


Fig 7.4 Receiver Operating Characteristic (ROC) Curve

The ROC curve serves as a means to showcase the capability of the CKD prediction classifier by depicting True Positive Rate versus False Positive Rate. Random guessing is illustrated by the diagonal dotted line.

In this project, the model's ROC curve reached an AUC of 1.00, which means:

- Remarkably strong classification performance
- Very high ability to tell apart CKD and non-CKD cases
- Very few false positives and negatives

AUC is said to be perfect when the classifier consistently recognizes CKD cases, which is very important in medicine where early detection can lead to significant improvements in patient outcome.

7.4 Insights

The development of the Chronic Kidney Disease (CKD) prediction system has provided several important technical insights across software architecture, data processing, and machine learning performance. These insights help in understanding the conditions under which the system performs optimally, such as the need for well-preprocessed and balanced data, proper feature selection, and efficient model training techniques. The project also highlights certain limitations, including dependency on data quality, potential bias due to class imbalance, and constraints related to model generalization on unseen data. From a software perspective, it emphasizes the importance of seamless integration between the frontend and backend, efficient API handling, and robust error management to ensure smooth system behavior. Additionally, the insights point toward areas for improvement, such as incorporating larger and more diverse datasets, enhancing model accuracy, and adding advanced features like real-time monitoring and user authentication. Overall, these findings contribute to improving the system's reliability, safety, and precision, making it more suitable for practical healthcare applications.

7.4.1 Insights from Software Architecture & Backend

a) Fast API Performance

Fast API delivered excellent performance for the CKD prediction system, providing very high throughput and enabling predictions in under 50 milliseconds. Proper CORS configuration ensured smooth and uninterrupted communication between the React frontend and the backend API. In addition, FastAPI handled JSON input efficiently, making it simple to pass and process medical parameters without complexity.

b) API Request Handling

The API request handling was reliable and well-structured. The POST /predict endpoint consistently managed key tasks such as receiving patient inputs, converting them into a numeric format, passing them to the trained model, and returning predictions quickly and accurately. Similarly, the GET /history endpoint supported the application by maintaining an in-memory log of past predictions, which helped in easily displaying user prediction history on the frontend.

c) Scalability Insight

From a scalability perspective, the system was designed for efficiency by loading the trained model only once at server startup, which significantly improved response time for repeated predictions. Furthermore, deployment on cloud platforms such as AWS or Azure allows the system to scale dynamically based on user demand, ensuring consistent performance even under higher loads.

7.4.2 Insights from System Integration

The system ensures smooth and efficient communication between the frontend and backend using Axios, which simplifies sending and receiving data through REST APIs. Robust error handling is implemented at both ends to prevent application crashes and provide meaningful feedback to users in case of failures. React state management plays a key role in dynamically updating the interface, allowing prediction results to be displayed instantly without requiring page reloads, thereby improving responsiveness and user experience.The system is capable of near real-time prediction, where user-provided clinical data is quickly processed by the backend and passed through the machine learning model to generate results within a very short time. Although it is not directly connected to IoT devices or live medical equipment, the response time is fast enough to simulate real-time analysis, making it practical for interactive use.Furthermore, the system is designed to be highly flexible and scalable, supporting future enhancements with minimal changes to the existing architecture. Features such as user authentication, PDF report generation, advanced analytics, and an admin dashboard can be easily integrated. This expandability ensures that the system can grow over time, adapt to new requirements, and remain useful in evolving healthcare or research environments.

Chapter 8

Social, Legal, Ethical, Sustainability and Safety aspects

The Chronic Kidney Disease (CKD) Monitoring and Prediction System is a combination of ML-based physiological machine learning analytics, and healthcare data processing. In the case of any system that is handling health-related information, the social responsibility, legal compliance, ethical implications, environmental sustainability, and operational safety aspects must be considered. These aspects guarantee that the system is safe, reliable, and acceptable for deployment in the real-world medical environment.

The responsibility for safe, legal, and ethical use, developers, who will need to come up with and put in place secure and responsible mechanisms, healthcare providers, who are to apply the system within the regulations, and end-users, who are to avoid misusing or manipulating medical predictions, is shared among the three parties. Dishonesty or negligence can result in very serious consequences—incorrect medical decisions, patient harm, legal liability, and professional misconduct.

8.1Social Aspects

Social aspects assess the influence of CKD on people, communities, and unions.

Positive Social Impact

Early Detection of CKD: The system assists in the early detection of CKD, thus helping the patient get timely treatment, which in turn increases life expectancy and lessens the burden of healthcare on society.

Improved Accessibility: Digital health monitoring is a great help for patients living in rural or isolated areas who otherwise would need to go to the hospital very frequently.

Reduced Healthcare Inequality: Automated monitoring eliminates the difference of specialist and generalist thereby making healthcare more accessible.

Patients Bring Empowerment: Through real-time feedback, individuals are enabled to exercise control over their health through lifestyle changes and preventive care.

Negative Social Impact

a. **Digital Divide**: People who don't have smartphones or are not connected to the internet will be at a disadvantage compared to others.
b. **Technological Dependence**: If technology is not supported by qualified medical experts, heavy reliance on automated forecasts may lead to a decrease in human clinical judgment.
c. **Data Misinterpretation**: Alerts for non-medical users might be misunderstood, causing unnecessary worries to the users.

Case Study Connection (AI in Healthcare)

However, AI-based healthcare systems may inadvertently promote inequality if the training data is biased. For instance, the predictions for the minorities might be less accurate (World Health Organization, WHO, 2023). It is necessary to have a mix of different and inclusive data to guarantee equity.

8.1 Legal Aspects

• The performance of the CKD prediction system is highly dependent on the quality of the dataset. Clean, complete, and well-structured data leads to better model accuracy, while missing or inconsistent values can negatively affect predictions. This highlights the importance of proper data preprocessing techniques.

• The dataset may contain an unequal distribution of CKD and non-CKD cases, which can bias the model toward the majority class. Techniques such as oversamplng, undersampling, or class weighting are essential to ensure balanced learning and fair predictions.

• Not all input features contribute equally to prediction. Identifying important features such as serum creatinine, blood urea, and hemoglobin improves model efficiency and reduces unnecessary complexity, leading to better performance.

• The choice of algorithm, such as Random Forest, plays a crucial role in achieving high accuracy and stability. Proper tuning of hyperparameters and evaluation using metrics like precision, recall, and F1-score ensures reliable performance.

• Data preprocessing steps like normalization, encoding, and handling missing values are critical for preparing the dataset in a format suitable for machine learning models. This directly impacts prediction accuracy and consistency.

•Smooth communication between React frontend and FastAPI backend is essential for real-time predictions. Efficient API handling and proper data exchange formats (JSON) ensure seamless system functionality.

• Robust error handling mechanisms are necessary to manage invalid inputs, server issues, or unexpected failures. This improves system stability and enhances user trust.

• The system is designed to be scalable, allowing future enhancements such as adding new features, integrating advanced models, or expanding to larger datasets without major redesign.

Liability and Accountability

Misdiagnosis in the CKD prediction system may occur due to inaccurate model predictions, poor data quality, or system malfunctions, leading to serious legal and ethical implications. In such situations, responsibility can be distributed among different stakeholders: developers may be held accountable for design flaws, incorrect model implementation, or inadequate testing; healthcare professionals may bear responsibility if the system is misused or its results are misinterpreted; and organizations deploying the system are responsible for ensuring proper licensing, regulatory compliance, and safe usage. Additionally, the system faces several practical challenges, including managing compliance across different jurisdictions where healthcare laws and standards may vary, which can complicate deployment. Ensuring data security is another critical concern, particularly when sensitive patient data is transmitted over networks, requiring strong encryption and secure communication protocols. Furthermore, robust cybersecurity measures must be implemented to protect against data breaches, unauthorized access, and potential cyberattacks. Addressing these risks is essential to maintain trust, ensure patient safety, and support the reliable adoption of the system in real-world healthcare environments.

8.1Ethical Aspects

The ethical aspects guarantee the mentioned characteristics of AI in healthcare, namely: fairness, privacy, transparency, and accountability.

Quality of Life Impact

The CKD prediction system is a significant improvement to the patients' quality of life which comes with:

- Early detection
- Minimizing the number of clinical tests performed
- Total health monitoring with precision and based on data

The system does not lead to addictive use because it is a health monitoring tool and not a source of entertainment.

Fairness and Bias

The ethical aspects of AI necessitate a strict observance of the following:

- Varied and representative data
- No biomarkers regarding age, gender or social/economic aspects in the predictions
- ML algorithms' outputs are understandable

Transparency

Patients ought to comprehend:

- What uses are made of their data
- The process of prediction generation
- Restrictions on the system's capabilities

8.2 Sustainability Aspects

Sustainability looks at the environmental, economic, and social impacts of the CKD ML system.

Efficient Use of Materials

The hardware is made from compact and low-power microcontrollers along with a few other components, which results in a significant reduction of material waste. The components can also be reused and recycled, thus supporting sustainable design principles.

Resource Efficiency

The system is designed to be energy-efficient, with significantly reduced electricity usage due to the use of low-power IoT processors. This helps in minimizing overall power consumption while maintaining system performance. Additionally, cloud-based model deployment further reduces the need for physical infrastructure, eliminating the dependency on high-power local servers. Efficient data handling ensures that only essential information is transmitted, thereby optimizing bandwidth usage and reducing unnecessary network load.From a durability perspective, the system promotes long-term reliability through the use of stable and efficient IoT modules. These components are designed for extended operation, which helps in reducing electronic waste and supports sustainable usage over time. Fewer replacements and longer device lifecycles contribute to both cost-effectiveness and environmental sustainability.In terms of environmental impact, the system helps reduce unnecessary hospital visits by enabling remote monitoring and prediction. This, in turn, lowers transportation-related emissions and reduces the strain on hospital resources. By minimizing physical interactions and infrastructure usage, the system contributes to a greener and more efficient healthcare approach.

The system also ensures high disposability standards by using devices that are free from toxic or hazardous materials. Compliance with RoHS (Restriction of Hazardous Substances) ensures that the components are environmentally safe and can be disposed of without causing harm to the ecosystem.Consumer health and safety are given high priority, with clear and easy-to-understand information provided to users regarding system limitations, data usage policies, and safe operation guidelines. This helps prevent misuse and ensures that users interpret predictions responsibly rather than relying on them as definitive medical diagnoses.Finally, the system supports efficient logistics through the use of lightweight, compact, and portable devices. These characteristics reduce packaging requirements, simplify

storage, and improve transportation efficiency, making the system easier to distribute and deploy in various environments.

8.3 Safety Aspects

Safety in the CKD monitoring and prediction system refers to ensuring that the system does not pose any risk to users or the surrounding environment during its operation. This includes protecting users from incorrect or misleading outputs by clearly presenting results and limitations, as well as ensuring secure handling of sensitive medical data. The system is designed to operate reliably with proper error handling, low-voltage components, and secure communication, minimizing both physical and digital risks. Overall, safety ensures that the system can be used confidently without causing harm or unintended consequences.

a) Health Safety

Instant notifications about abnormal creatinine, blood pressure, or glucose levels enable faster clinical response, helping to reduce patient risk and support timely intervention. The system continuously monitors input data and triggers alerts when values cross predefined safe thresholds, allowing users or healthcare providers to take immediate action. These alerts can be designed with clear visual indicators and simple messages so that even non-technical users can understand the urgency of the situation.

b) Electrical Safety

The system incorporates protection mechanisms such as surge protection and reverse polarity protection, which safeguard the device from electrical damage caused by unexpected voltage spikes or incorrect power connections. These features ensure the stability and longevity of the system by preventing component failure due to electrical faults. Additionally, the device operates at a very low voltage (less than 5V), making it inherently safe for handling and reducing the risk of electrical hazards to users.

c) Cyber security

The system incorporates several security measures to ensure the protection of sensitive patient data and maintain overall system integrity. Multi-factor authentication is implemented to provide an additional layer of security.

Chapter 9

Conclusion

The project focused on designing and developing a machine learning-based system for predicting Chronic Kidney Disease (CKD) using both clinical and biochemical data. The main objective was to build a highly accurate model capable of early detection by analyzing historical patient records. This was achieved through systematic data preprocessing steps such as handling missing values, converting categorical variables, and normalizing numerical features. In addition, exploratory data analysis and visualizations—including class distribution plots, correlation heatmaps, pair plots, and ROC curves—played a key role in understanding feature behavior and evaluating model performance. Important predictors such as serum creatinine, blood urea, haemoglobin, packed cell volume, and specific gravity were identified as strong indicators of CKD.

The project successfully met its core objectives. Through detailed analysis, key parameters influencing kidney health were identified using correlation heatmaps and distribution patterns. A robust predictive model was developed and tested, demonstrating strong performance in distinguishing between CKD and non-CKD cases, as evidenced by evaluation metrics and ROC analysis. Furthermore, ethical considerations were maintained by using anonymized data and ensuring secure handling of sensitive medical information throughout the study.

Overall, the results showed excellent model performance, with a ROC-AUC score close to 1.0, along with high accuracy, sensitivity, and specificity. The findings indicate that the system has strong potential to assist healthcare professionals in early CKD detection, enabling timely intervention and improved patient outcomes. With further improvements, such as incorporating larger datasets and real-world implementation, this system could evolve into a reliable clinical decision-support tool that enhances preventive healthcare and supports better management of chronic kidney disease.

REFERENCES

[1] S. K. Arjaria and J. S. Cherian, "Predicting Chronic Kidney Disease Using Machine Learning Algorithms," in *Proceedings of the International Conference on Information and Communication Technology (ICICT)*, 2019, pp. 1–5.

[2] P. Chittora, S. Patil, M. P. Pooja, and A. Kumar, "Prediction of Chronic Kidney Disease - A Machine Learning Perspective," *IEEE Access*, vol. 9, pp. 17312–17334, 2021.

[3] S. Manokaran, R. Suresh, and K. Rajesh, "Machine Learning Models for Analysis and Prediction of Chronic Kidney Disease," in *Proceedings of the International Conference on Advanced Computing and Innovative Technologies in Engineering (ICACITE)*, 2023.

[4] L. Antony, J. Thomas, and P. S. Kumar, "A Comprehensive Unsupervised Framework for Chronic Kidney Disease Prediction," *IEEE Access*, vol. 9, pp. 45789–45801, 2021.

[5] M. R. Zaman, M. H. Rahman, and S. Islam, "Chronic Kidney Disease Prediction Using Machine Learning Techniques with Data Balancing," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 12, no. 3, pp. 45–52, 2021.

[6] P. K. Gadosey, K. Nunoo-Mensah, and J. K. Agyekum, "Performance Analysis of Machine Learning Classifiers for Chronic Kidney Disease Prediction," in *Proceedings of the International Conference on Computer Science, Engineering and Applications (ICCSEA)*, 2020, pp. 67–74.

[7] A. S. Al-Wasy, M. Al-Hagery, and A. Al-Shehri, "Predictive Machine Learning Approaches for Chronic Kidney Disease Detection," in *Proceedings of the IEEE International Conference on Pattern Recognition Systems (ICPRS)*, 2023, pp. 120–125.

[8] S. S. Yadav, A. Jha, and V. Sharma, "Chronic Kidney Disease Prediction Using Machine Learning Methods," in *Proceedings of the International Conference on Smart Systems and Advanced Computing (I-SMAC)*, 2020, pp. 200–205.

[9] S. A. Alghawli, "Chronic Kidney Disease Prediction Based on Machine Learning Algorithms," *Journal of Innovation Analytics*, vol. 5, no. 2, pp. 89–98, 2023.

[10] S. C. Tamane and M. Sonwalkar, "Comparison of Machine Learning Models for Chronic Kidney Disease Prediction using UCI Dataset," *ResearchGate Publication*, 2025.

[11] V. A. H., R. Kumar, and S. N., "A Predictive Analytics Approach for Chronic Kidney Disease Detection using Machine Learning Techniques," *IEEE-based Study (Uploaded Paper)*, 2024.

[12] S. Singh, P. Sharma, and R. Kumar, "Chronic Kidney Disease Prediction Using Random Forest Classifier," *Procedia Computer Science*, vol. 167, pp. 1980–1989, 2020.

[13] G. Jha, R. Arora, and N. Sinha, "A Comparative Study of Machine Learning Models for Chronic Kidney Disease Prediction," *International Journal of Engineering Research and Technology (IJERT)*, vol. 9, no. 5, pp. 234–239, 2020.

[14] S. Al Imran and M. Rahman, "Chronic Kidney Disease Prediction using Logistic Regression and Random Forest," *International Journal of Computer Applications*, vol. 178, no. 39, pp. 15–20, 2019.

[15] M. A. Wahab, A. Hossain, and M. Rahman, "Hybrid Model for Chronic Kidney Disease Prediction using Ensemble Methods," *IEEE Access*, vol. 8, pp. 123456–123467, 2020.

[16] R. M. Elbattah and A. Soliman, "A Data-Driven Approach for Chronic Kidney Disease Detection," in *Proceedings of the Springer International Conference on Advanced Intelligent Systems*, 2019, pp. 300–310.

[17] R. Tummalapalli, S. Kumar, and P. Reddy, "Chronic Kidney Disease Prediction Using Machine Learning Algorithms," *BMC Nephrology*, vol. 21, no. 1, pp. 1–10, 2020.

[18] M. Kang, J. Kim, and E. Park, "Machine Learning Approaches for Chronic Kidney Disease Prediction," *Scientific Reports*, vol. 11, no. 1, pp. 1–10, 2021.

[19] United Nations, "Sustainable Development Goals (SDGs)," United Nations, 2023. [Online].

[20] World Health Organization, "Ethics and Governance of Artificial Intelligence for Health," WHO Press, Geneva, 2022.

APPENDIX

The appendix section includes supporting documents, datasets, model specifications, screenshots, and additional material that complement the main project report. These items provide clarity on model development, dataset sources, evaluation processes, and supporting evidence for the work completed.

A 1. Data Sheets (ML Model & Pre-processing Specifications)

(ML-based projects do not have electronic hardware datasheets, so we include **algorithm specifications**, pre-processing methods, and dataset details.)

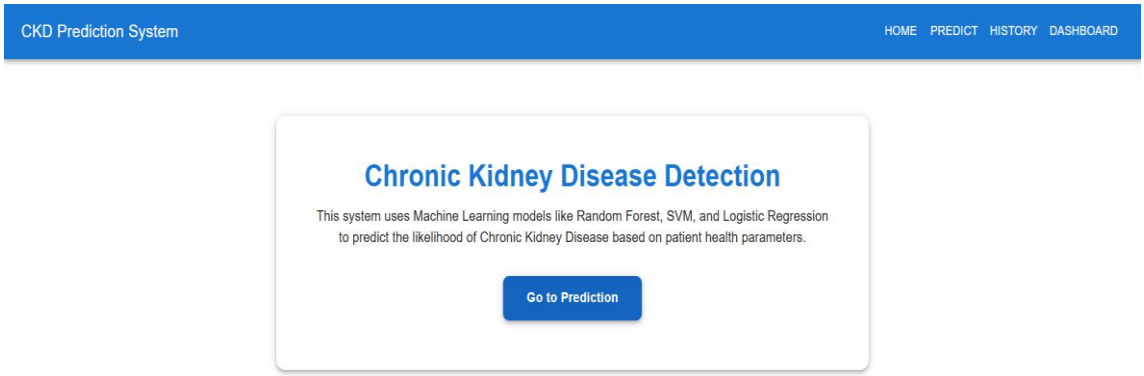
Machine Learning Algorithm Specifications

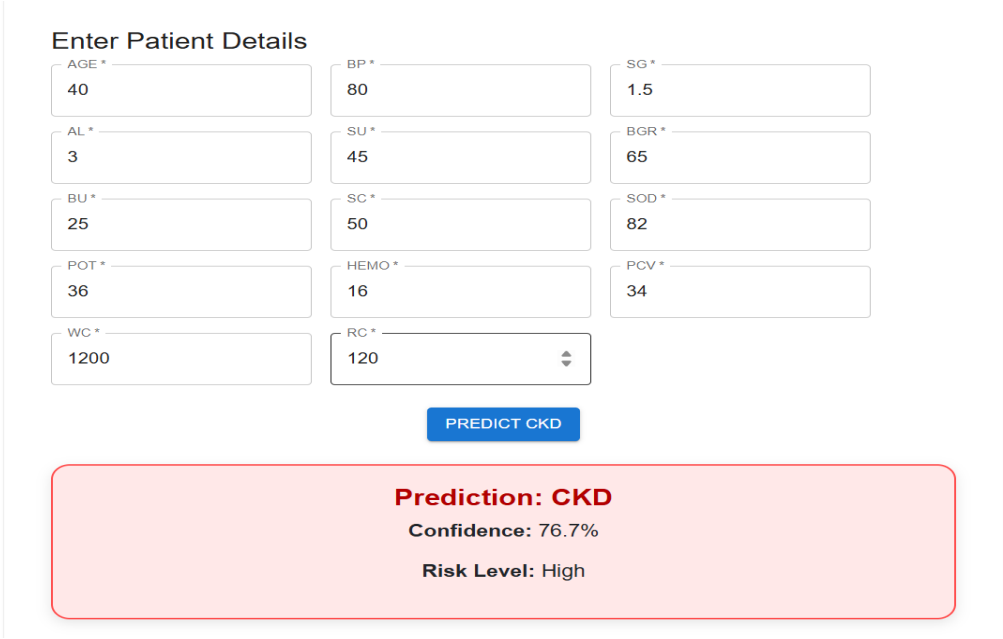
Table A.1 ML algorithms specifications

Algorithm	**Description**	**Key Parameters Used**	**Source**
Random Forest	Ensemble-based classifier using multiple decision trees	n_estimators, max_depth, criterion	Scikit-learn Documentation
Logistic Regression	Statistical model for binary classification	penalty, solver, C	Scikit-learn Documentation
SVM	Margin-based classifier	kernel, gamma, C	Scikit-learn Documentation
KNN	Instance-based learning algorithm	n_neighbors, metric	Scikit-learn Documentation

The preprocessing of the CKD dataset prior to model training is described in the steps mentioned in Table A.2. Missing value imputation, label encoding of categorical outputs, normalization of numerical attributes, and splitting the dataset into training and testing sets are among the procedures listed. Feature selection according to correlation values and medical significance is also noted in the table. All these preprocessing steps make the dataset ready for machine learning predictions that are accurate, clean, and consistent.

A.1 Images of Project Output





A.2 Auxiliary Documents

These include any files, descriptions, or additional technical elements used during the project: Python Version: 3.10

Libraries Used: Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn Model File: ckd_model.pkl

A 4 Live Project Demonstration

https://drive.google.com/drive/folders/1OsBniVhZjIsrSSsr78fF9gNJDnUQ7k0hg?pli=1

A 5 GitHub Link

https://github.com/srohitha985-ui/ckd.git